

*Communauté Économique et Monétaire
de l'Afrique Centrale
(CEMAC)*



*Institut Sous-régional de Statistique
et d'Économie Appliquée*

Organisation Internationale

BP : 294 Yaoundé - Tél : 222 220 134

Cameroon AI Enthousiaste



*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du
diplôme d'Ingénieur Statisticien Économiste (ISE)*

OPTION : Data Science et Marketing

CONCEPTION D'UN SYSTEME DE RECOMMANDATION HYBRIDE POUR LE MATCHING EMPLOI-COMPETENCES AU CAMEROUN PAR INTEGRATION DES LLMS ET DES GRAPHES DE CONNAISSANCES

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Sous l'encadrement de :

Dr. FOUTSE Yuehgoh

Présidente & Directrice exécutive adjointe, PhD Computer Science & Data Scientist

Année académique 2025-2026

Dédicace

À mes parents... (Max 1 page)

Remerciements

Sommaire

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	vi
Liste des tableaux	vii
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
 I Cadre théorique et conceptuel	 5
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	6
1.1 Marché de l'emploi, information imparfaite et appariement des compétences	6
1.2 Les fondements des LLMs	9
1.3 Généralité sur le fine-tuning des modèles de LLMs	14
1.4 Généralité sur des BD vectorielles et orientées graphes	16
1.5 Généralité sur les systèmes de recommandation	20
1.6 Génération Augmentée par Récupération (RAG) et GraphRAG	23
1.7 Agentic RAG	25
 2 Approche méthodologique et données	 28
2.1 Positionnement méthodologique	28
2.2 Sources de données et rôle dans le système	29
2.3 Préparation et normalisation des données	30
2.4 Alignement des référentiels métiers et formations	31
2.5 Choix du modèle d'embeddings et fine-tuning	31
2.6 Indexation vectorielle dans PostgreSQL et pgvector	32

2.7	Construction du graphe de connaissances Neo4j	32
2.8	Score hybride et logique de skill gap	32
2.9	Architecture Agentic RAG et orchestration LangGraph	33
2.10	Text2Cypher et rôle du LLM	33
2.11	Interfaces et traçabilité	34
2.12	Protocole d'évaluation prévu	34
2.13	Limites méthodologiques et précautions d'interprétation	34
2.14	Synthèse de l'approche méthodologique	35
II	Présentation des résultats	36
3	Implémentation du système de recommandation	37
3.1	Vue d'ensemble de l'implémentation	37
3.2	Implémentation de la chaîne ETL	38
3.3	Implémentation du fine-tuning SentenceTransformer	39
3.4	Implémentation de la base vectorielle pgvector	39
3.5	Implémentation du graphe de connaissances Neo4j	40
3.6	Implémentation du moteur de recommandation hybride	41
3.7	Implémentation du pipeline Text2Cypher	41
3.8	Implémentation du workflow agentique LangGraph	42
3.9	Implémentation du LLM final et du critic	43
3.10	API, CLI et interface Streamlit	43
3.11	Diagnostic d'intégration	44
3.12	Synthèse du chapitre	45
4	Évaluation de la performance du système	46
4.1	Objectif et logique générale de l'évaluation	46
4.2	Données et protocole expérimental	46
4.3	Évaluation du modèle d'embeddings	47
4.4	Ablation pgvector seul contre pgvector + graphe	48
4.5	Analyse du reranking par le graphe	49
4.6	Performance opérationnelle et stabilité	50
4.7	Interprétation par rapport aux hypothèses	51
4.8	Limites de l'évaluation	51
4.9	Synthèse du chapitre	51
5	Discussion et recommandations	52
	Conclusion générale	I
	Bibliographie	I
	Bibliographie	II

Liste des sigles et abréviations

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory
MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

Liste des tableaux

2.1	Correspondance entre objectifs spécifiques et choix méthodologiques	29
2.2	Sources de données mobilisées et utilisation méthodologique	30
3.1	Organisation opérationnelle des modules implémentés	38
3.2	Entités indexées dans pgvector au moment du diagnostic	40
3.3	Principales fonctions du graphe dans le système	41
3.4	Outils exposés à l'agent LangGraph	43
3.5	Modes d'accès au système implémenté	44
3.6	Résultat du diagnostic local d'intégration	44
4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	47
4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	48
A.1	Labels de nœuds et relations structurantes du graphe	V
A.2	Volume d'entités indexées dans la base vectorielle pgvector	VI

Table des figures

1.1	Évolution chronologique des LLMs (2013-2024)	10
1.2	Mécanisme de self-attention	13
1.3	Architecture complète du Transformer	14
1.4	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	25
3.1	Workflow agentique implémenté dans LangGraph	42
4.1	Comparaison des modèles d'embeddings sur le jeu de test	48
4.2	Effet de l'enrichissement graphe sur les métriques de retrieval	49
4.3	Distribution des déplacements de rang après reranking par le graphe	50

Avant-propos

L’Institut Sous-régional de Statistique et d’Économie Appliquée (ISSEA), institution spécialisée de la CEMAC créée en 1984, a pour mission principale la formation de cadres supérieurs en statistique, économie et data science, capables de répondre aux enjeux contemporains de décision fondée sur les données. Dans ce cadre, l’ISSEA propose plusieurs filières de formation, parmi lesquelles figure celle des Ingénieurs Statisticiens Économistes (ISE), alliant rigueur mathématique, modélisation économétrique et outils modernes d’analyse de données (Data Science).

Arrivés à un niveau avancé de leur formation, les élèves Ingénieurs Statisticiens Économistes sont tenus d’effectuer un stage professionnel d’une durée minimale de trois (03) mois. Ce stage constitue une phase essentielle du cursus, permettant de confronter les connaissances théoriques acquises à des problématiques réelles, tout en favorisant l’acquisition de compétences opérationnelles en environnement professionnel. Il est sanctionné par la rédaction d’un mémoire professionnel, soutenu devant un jury.

C’est dans ce cadre que nous avons effectué un stage du 9 mars 2026 au 30 juin 2026 au sein de KmerAI, où nous avons été intégrés à la Direction du Développement et de la Recherche. Cette immersion nous a permis de travailler sur une problématique appliquée située au croisement de l’analyse de données, de l’intelligence artificielle générative, du traitement automatique du langage naturel et de l’aide à la décision : l’amélioration du matching entre offres d’emploi, profils de candidats et compétences recherchées sur le marché camerounais.

Le présent mémoire s’inscrit dans cette dynamique et porte sur le thème : « Conception d’un système de recommandation hybride pour le matching emploi-compétences au Cameroun par intégration des LLMs et des graphes de connaissances ». Il vise principalement à concevoir une architecture capable de structurer les données d’offres et de profils, d’exploiter les référentiels métiers et formations, puis de produire des recommandations plus explicables que les approches fondées uniquement sur des mots-clés. Plus spécifiquement, il s’agit de normaliser les données disponibles, d’aligner les métiers et compétences avec les référentiels ESCO, MEPC et NCF, d’adapter un modèle d’embeddings au domaine emploi-compétences, de construire un graphe de connaissances sous Neo4j et de poser les bases d’un moteur de recommandation hybride intégrant similarité sémantique, relations de graphe et analyse du *skill gap*.

Ce travail répond à un double enjeu. Sur le plan scientifique et méthodologique, il cherche à montrer comment les modèles de représentation sémantique, les bases vectorielles et les graphes de connaissances peuvent être combinés pour traiter un problème réel d’appariement sur le marché du travail. Sur le plan opérationnel, il propose une démarche reproductible susceptible d’aider KmerAI à développer des outils de recommandation, d’orientation et de montée en compétences adaptés aux besoins des candidats, des recruteurs et des acteurs de la formation.

Résumé

Mots-clés : Économétrie, Statistique, Développement...

Abstract

INTRODUCTION GÉNÉRALE

Contexte et justification

Le fonctionnement du marché de l'emploi constitue un déterminant central de la performance économique et sociale d'un pays. Au Cameroun, ce marché reste marqué par des déséquilibres structurels persistants, notamment l'inadéquation entre les compétences disponibles et les besoins des entreprises, la forte informalité et les difficultés d'insertion professionnelle des jeunes. Les données disponibles montrent que le chômage des jeunes demeure une réalité importante, avec un taux estimé à 6,6% en 2024 et 6,5% en 2025 selon les estimations modélisées de l'OIT relayées par la Banque Mondiale et la base FRED. Par ailleurs, plusieurs travaux sur le Cameroun soulignent que le problème ne tient pas uniquement au manque d'emplois, mais aussi à un mismatch entre formation, compétences et exigences du marché.

Dans ce contexte, la question du matching emploi-compétences devient centrale. En théorie, un marché du travail efficient devrait permettre d'associer rapidement les profils disposant des compétences requises aux postes vacants correspondants. En pratique, plusieurs frictions informationnelles persistent : asymétrie d'information entre recruteurs et candidats, faible structuration des profils de compétences, hétérogénéité des offres d'emploi et faiblesse des systèmes d'intermédiation comme le FNE et certains agences de recrutement. Ces dysfonctionnements conduisent à une situation paradoxale où coexistent des postes non pourvus et des chercheurs d'emploi, traduisant une inefficacité allocative du marché du travail.

Par ailleurs, l'essor des technologies numériques et de l'intelligence artificielle ouvre des perspectives nouvelles pour améliorer l'appariement entre offres et profils. Dans cette dynamique, KmerAI se positionne comme une plateforme camerounaise dédiée à la promotion de l'intelligence artificielle, à la vulgarisation de ses usages et à l'accompagnement des étudiants, chercheurs et entreprises dans l'adoption de solutions innovantes adaptées au contexte local. Ses missions consistent notamment à former aux bases de l'IA, à offrir un espace de partage d'expériences et à encourager la collaboration autour de cas d'usage appliqués à des secteurs stratégiques tels que la santé, l'agriculture, le transport, l'éducation et le marketing.

Cette orientation est particulièrement pertinente pour le marché de l'emploi camerounais, où les données liées aux offres, aux CV et aux compétences restent souvent dispersées et peu exploitées. Une approche fondée sur l'intelligence artificielle peut permettre d'analyser plus efficacement ces données hétérogènes afin de proposer des recommandations d'emploi plus pertinentes, plus rapides et plus personnalisées.

Problématique

Le marché de l'emploi camerounais est marqué par une forte asymétrie d'information, une hétérogénéité des profils professionnels et des offres d'emploi, ainsi qu'une faible capacité des outils classiques à évaluer les écarts réels de compétences. Les systèmes fondés sur des mots-clés ou des filtres statiques ne parviennent pas à détecter les compétences implicites, les équivalences sémantiques et les trajectoires de montée en compétences.

Par ailleurs, si des référentiels structurants existent notamment la Nomenclature Camerounaise des

Métiers (NCM 2013), le référentiel européen ESCO et la Nomenclature Camerounaise des Formations (NCF), leur intégration opérationnelle dans les plateformes et outils d'appariement reste encore limitée. Ils sont peu exploités pour aligner les intitulés locaux de métiers sur des standards internationaux, harmoniser les compétences attendues, qualifier les niveaux et domaines de formation, et rendre les correspondances profil-offre plus transparentes et comparables. Il en résulte un décalage entre la richesse potentielle de ces référentiels et leur usage effectif dans les systèmes d'information dédiés à l'emploi et à l'orientation.

Dans ce contexte, la question centrale n'est plus seulement d'apparier un profil à une offre, mais de concevoir un système capable (i) d'exploiter conjointement les référentiels NCM, ESCO et NCF pour structurer les métiers, les compétences et les formations, (ii) de mesurer finement le *skill gap* entre un candidat et un poste cible, et (iii) de proposer un parcours de montée en compétences compatible avec le niveau, le domaine de formation et les trajectoires de qualification du candidat. La question principale de recherche peut ainsi être formulée comme suit :

Comment concevoir un système de recommandation hybride, fondé sur les LLMs, les graphes de connaissances, les embeddings vectoriels et le filtrage collaboratif, capable de mesurer le skill gap entre un profil et une offre d'emploi, puis de recommander un parcours de montée en compétences cohérent avec la nomenclature des formations, le niveau du candidat et le métier visé pour les membres de la communauté KmerAi ?

Cette question générale se décline en plusieurs questions de recherche sous-jacentes :

- **Q1** : Comment identifier et formaliser, dans le contexte camerounais, les compétences, métiers, formations et relations pertinentes pour un matching emploi-compétences fiable et opérationnel à partir des référentiels NCM, ESCO et NCF ?
- **Q2** : Dans quelle mesure l'intégration d'un graphe de connaissances améliore-t-elle la qualité et la précision du matching par rapport aux approches purement vectorielles ?

Objectifs de l'étude

L'objectif général de cette étude est de concevoir et d'évaluer un système de recommandation hybride emploi-compétences capable d'aider à la décision dans le recrutement et la montée en compétence. Le système vise à rapprocher les profils de candidats et les offres d'emploi en combinant la recherche sémantique, les référentiels métiers et formations, le graphe de connaissances et une logique explicable de *skill gap*. Il ne s'agit donc pas seulement de retrouver les offres textuellement proches d'un profil, mais de produire un verdict opérationnel : identifier les offres immédiatement accessibles, distinguer celles qui nécessitent une montée en compétence, signaler les profils à conserver en vivier et expliciter les compétences à développer.

Plus spécifiquement, il sera question de :

- normaliser et aligner les offres d'emploi et profils candidats avec les référentiels ESCO, MEPC et NCF afin de produire des variables structurées — intitulé de poste, compétences, niveau NCF, secteur, localisation — exploitables par le système de matching ;
- adapter par *fine-tuning* contrastif un modèle *SentenceTransformer* au domaine emploi-compétences à partir de paires offre-profil, puis indexer les représentations produites dans une base vectorielle *pgvector* pour le retrieval sémantique ;

- construire un graphe de connaissances sous Neo4j modélisant les relations entre candidats, offres, compétences, métiers, secteurs et niveaux NCF, et concevoir un score hybride combinant similarité sémantique, couverture de compétences, compatibilité NCF et alignement sectoriel;
- orchestrer, via un workflow *LangGraph* multi-étapes, l'ensemble du pipeline de recommandation — retrieval sémantique, enrichissement graphe, analyse de *skill gap*, scoring hybride, critique et replanification — afin de produire des verdicts interprétables (profil prêt, montée en compétence, vivier, hors cible) accompagnés d'une trajectoire de formation, et d'évaluer la qualité du retrieval par les métriques Recall@K, NDCG@K, MRR@K et MAP@K.

Intérêt de l'étude

L'intérêt d'une telle approche est renforcé par les limites des méthodes classiques de recrutement, souvent fondées sur des critères statiques ou sur des mots-clés, qui ne captent pas toujours la richesse des parcours et des compétences. En combinant différentes logiques de recommandation, un système hybride peut mieux prendre en compte les dimensions explicites et implicites des profils candidats, tout en améliorant la qualité des correspondances proposées. Cela rejoint la mission de KmerAI, qui vise à favoriser l'appropriation locale de l'IA pour résoudre des problèmes concrets du développement camerounais.

Hypothèses de recherche

Afin de répondre à la problématique, cette étude repose sur les hypothèses suivantes.

- **H1** : L'intégration conjointe de la NCM, d'ESCO et de la NCF dans un référentiel unifié métiers-compétences-formations améliore la structuration et les performances d'appariement profil-offre, par rapport à une approche sans ces référentiels;
- **H2** : La modélisation du marché de l'emploi sous forme de graphe de connaissances (Neo4j), incluant explicitement les niveaux et domaines de formation de la NCF, améliore la qualité du matching par rapport à une approche purement vectorielle;
- **H3** : Le *fine-tuning* d'un modèle de représentation sémantique sur le domaine emploi-compétences améliore la qualité des embeddings et la similarité sémantique entre profils et offres, par rapport au même modèle non adapté;
- **H4** : La combinaison d'un RAG vectoriel, d'un GraphRAG et d'un score de recommandation hybride améliore la pertinence et l'explicabilité des recommandations de *skill gap* et de parcours de montée en compétences, notamment lorsque ces parcours sont contraints par la nomenclature des formations.

Méthodologie de recherche

La démarche adoptée s'inscrit dans un cadre d'ingénierie des données combinant structuration des connaissances et modélisation hybride. Elle repose sur la collecte et la normalisation des données issues d'offres d'emploi sur les différents sites web et de profils, alignées sur la NCM 2013 afin d'assurer la cohérence sémantique. Un graphe de connaissances est ensuite construit sous Neo4j en intégrant les référentiels ESCO et NCM, à partir d'un processus d'extraction automatique de triplets à partir des données textuelles. Parallèlement, les descriptions sont vectorisées et indexées dans un espace sémantique permettant la mesure de similarité. Le moteur de recommandation combine ces différentes sources d'information à travers un score hybride intégrant similarité structurelle, similarité sémantique et signal collaboratif. Le système permet ainsi d'identifier les écarts de compétences entre profils et offres, et de générer

des recommandations sous forme de trajectoires d'apprentissage. L'évaluation repose sur des métriques standard telles que la précision@K, le rappel et le NDCG, ainsi que sur l'analyse de la cohérence des recommandations générées.

Plan du travail

Le mémoire est structuré en trois parties. La première présente le cadre théorique relatif aux systèmes de recommandation, aux modèles de langage et aux graphes de connaissances. La deuxième détaille l'approche méthodologique, incluant la modélisation du graphe et la conception du système hybride. La troisième analyse les résultats obtenus et discute les performances du modèle ainsi que ses implications dans le contexte du marché du travail camerounais.

Première partie

Cadre théorique et conceptuel

FONDEMENTS THÉORIQUES ET REVUE DE LA LITTÉRATURE SUR L'IA GÉNÉRATIVE ET LES SYSTÈMES DE RECOMMANDATION

Ce chapitre constitue la colonne vertébrale théorique du présent mémoire. Il vise à positionner ce travail dans le paysage scientifique actuel en passant en revue les avancées les plus significatives dans six domaines complémentaires et interdépendants : le fonctionnement du marché de l'emploi, les LLMs, les systèmes de recommandation, la théorie des graphes et les graphes de connaissances, la recherche sémantique et la RAG, les métriques d'évaluation, ainsi que les défis et limites identifiés dans la littérature.

1.1 Marché de l'emploi, information imparfaite et appariement des compétences

Le thème du matching emploi-compétences ne peut pas être réduit à un problème purement informatique. Il s'inscrit d'abord dans une problématique économique classique : comment un marché du travail met-il en relation des travailleurs hétérogènes, porteurs de compétences observables et non observables, avec des emplois eux-mêmes hétérogènes par leurs exigences techniques, organisationnelles et sectorielles ? Dans un cadre concurrentiel simplifié, l'ajustement entre l'offre et la demande de travail devrait s'opérer par les salaires et conduire à une allocation efficace des travailleurs vers les postes où leur productivité est la plus élevée. Cette représentation est toutefois insuffisante pour analyser les marchés du travail réels, particulièrement dans les économies où l'information sur les compétences, les postes disponibles et les trajectoires de formation est incomplète.

1.1.1 Le marché du travail comme marché d'appariement

La théorie moderne de la recherche d'emploi considère le marché du travail comme un marché d'appariement plutôt que comme un simple marché d'équilibre instantané. Les travailleurs ne trouvent pas immédiatement l'emploi correspondant à leurs compétences, et les entreprises ne rencontrent pas instantanément le candidat adapté à leurs besoins. La rencontre entre offre et demande implique donc des coûts de recherche, des délais, des incertitudes et des frictions informationnelles. Les modèles de recherche et d'appariement de Mortensen et Pissarides montrent que le chômage peut coexister avec des postes vacants, non pas uniquement parce que le niveau global de demande est insuffisant, mais aussi parce que le processus de rencontre entre travailleurs et entreprises est imparfait [1, 2].

Cette approche est particulièrement pertinente pour le contexte du présent mémoire. Un système de recommandation emploi-compétences cherche précisément à réduire ces frictions : il améliore la visibilité

des offres, structure les profils, compare les compétences détenues et requises, puis propose des correspondances classées. Autrement dit, il agit comme une technologie d'intermédiation. Sa valeur ne réside donc pas seulement dans sa performance algorithmique, mais dans sa capacité à améliorer l'efficacité du mécanisme d'appariement sur le marché de l'emploi.

1.1.2 Asymétrie d'information et signalement des compétences

Le marché de l'emploi est aussi marqué par une forte asymétrie d'information. Les candidats connaissent mieux que les recruteurs certaines dimensions de leurs compétences, de leur motivation et de leur capacité d'apprentissage. Inversement, les entreprises connaissent mieux que les candidats la réalité des tâches, les conditions de travail et les compétences effectivement valorisées dans le poste. Akerlof a montré que l'asymétrie d'information peut conduire à une sélection adverse, c'est-à-dire à une dégradation de la qualité moyenne des échanges lorsque les acteurs ne peuvent pas distinguer correctement les bons et les mauvais profils [3]. Sur le marché du travail, ce problème se traduit par une difficulté à identifier les candidats réellement compétents lorsque les informations disponibles sont incomplètes, non standardisées ou difficilement comparables.

Dans cette perspective, les diplômes, certifications, expériences professionnelles et portefeuilles de projets jouent un rôle de signal. La théorie du signalement de Spence explique que certains attributs observables permettent aux candidats de transmettre une information crédible sur leur productivité potentielle [4]. Toutefois, dans les métiers numériques et dans les environnements où les compétences évoluent rapidement, les signaux traditionnels deviennent incomplets. Un intitulé de diplôme ne suffit pas toujours à inférer la maîtrise effective de Python, de l'analyse de données, de la gestion de bases relationnelles ou de la conception d'API. Le recours à des référentiels de compétences comme la NCM et ESCO, combinés à des représentations sémantiques, permet alors de transformer des signaux dispersés en informations structurées et comparables.

1.1.3 Capital humain, compétences et inadéquation

La théorie du capital humain considère l'éducation, la formation et l'expérience comme des investissements qui accroissent la productivité individuelle et les revenus futurs [5]. Cette théorie justifie l'importance accordée aux compétences dans l'analyse du marché du travail. Cependant, posséder un capital humain ne garantit pas automatiquement une bonne insertion professionnelle. Encore faut-il que les compétences acquises correspondent aux compétences demandées. Le problème central devient alors celui de l'inadéquation, ou *mismatch*, entre formation, compétences et besoins productifs.

L'inadéquation peut prendre plusieurs formes. Elle peut être verticale lorsqu'un individu est surqualifié ou sous-qualifié pour un poste. Elle peut être horizontale lorsque le domaine de formation ou d'expérience ne correspond pas au contenu réel de l'emploi. Elle peut enfin être sémantique lorsque les mêmes compétences sont décrites par des termes différents dans les CV, les offres d'emploi et les référentiels métiers. C'est cette dernière forme qui justifie directement l'utilisation des LLMs, des embeddings et des graphes de connaissances : le système doit reconnaître que des formulations différentes peuvent renvoyer à des compétences proches, complémentaires ou substituables.

1.1.4 Transformation numérique et demande de compétences

Les travaux récents sur le changement technologique montrent que la demande de travail se transforme sous l'effet de l'automatisation, de la numérisation et de la polarisation des tâches. Autor souligne que les technologies ne remplacent pas simplement des emplois entiers ; elles modifient surtout la composition des tâches au sein des métiers [6]. Cette distinction est décisive pour un système de recommandation. Apparier un candidat à un métier ne suffit plus : il faut identifier les tâches et les compétences associées, puis mesurer l'écart entre le profil actuel et le profil cible.

Dans le contexte camerounais, cette logique est encore plus importante car les intitulés de postes peuvent être peu standardisés, les parcours professionnels hétérogènes et les données d'emploi souvent non structurées. Un moteur hybride combinant graphes de connaissances, embeddings vectoriels et LLMs permet de représenter plus finement les relations entre métiers, compétences, secteurs, formations et trajectoires de montée en compétence. Il répond ainsi à une limite fondamentale du marché de l'emploi : l'information utile existe souvent, mais elle est dispersée, ambiguë et difficilement exploitable sans structuration algorithmique.

Ainsi, la littérature économique fournit le socle théorique du présent travail. Les systèmes de recommandation ne sont pas seulement des outils techniques ; ils peuvent être analysés comme des dispositifs d'intermédiation visant à réduire les coûts de recherche, les asymétries d'information et l'inadéquation des compétences. Cette lecture permet de relier directement la contribution informatique du mémoire à une problématique économique : améliorer l'efficacité de l'appariement sur le marché du travail camerounais.

1.1.5 Compétences, formation et besoin de standardisation

La notion de compétence occupe une place centrale dans l'analyse contemporaine du marché du travail. Elle ne se limite pas à la possession d'un diplôme ou à l'accumulation d'années d'expérience ; elle renvoie à la capacité effective de mobiliser des connaissances, des savoir-faire, des attitudes professionnelles et des ressources cognitives dans une situation de travail donnée. Dans une économie où les métiers se transforment rapidement, l'employabilité dépend donc moins d'un titre scolaire isolé que d'un portefeuille de compétences actualisables, transférables et lisibles par les employeurs.

La formation intervient comme le mécanisme principal de constitution, d'actualisation et de reconversion de ce portefeuille. Dans la perspective du capital humain, elle accroît la productivité attendue des individus et améliore leurs perspectives d'insertion [5]. Mais cette relation n'est pas mécanique. Une formation peut être de bonne qualité sur le plan académique tout en restant faiblement alignée sur les besoins opérationnels du marché. Inversement, un candidat peut disposer de compétences utiles sans que celles-ci soient visibles dans son CV, parce qu'elles sont décrites avec des termes non standardisés, dispersées dans des expériences informelles ou absentes des nomenclatures utilisées par les recruteurs.

Ce point est décisif pour le Cameroun. Le problème du matching emploi-compétences ne vient pas seulement d'un déficit de formation ; il vient aussi d'un déficit de lisibilité. Les candidats décrivent leurs capacités avec des formulations hétérogènes, les entreprises expriment leurs besoins dans des langages métier variables, et les plateformes d'emploi exploitent souvent des mots-clés sans comprendre les équivalences entre compétences proches. Par exemple, « analyse de données », « data analytics », « exploitation statistique » et « reporting décisionnel » peuvent désigner des familles de compétences partiellement communes, mais un moteur lexical naïf peut les traiter comme des réalités séparées.

C'est précisément ici qu'une nomenclature de standardisation devient stratégique. Une nomenclature métiers-compétences fournit un langage commun pour nommer les professions, les activités, les compétences et les niveaux de qualification. La NCM permet d'ancrer l'analyse dans le contexte camerounais, tandis que le référentiel européen ESCO propose une structuration fine des relations entre professions, aptitudes, compétences et qualifications [7]. L'intérêt n'est pas de remplacer les intitulés locaux par des catégories étrangères, mais de construire une table de correspondance entre les formulations observées dans les données et des concepts stabilisés.

Une telle standardisation produit trois gains analytiques. Premièrement, elle améliore la comparabilité : deux profils peuvent être comparés même s'ils utilisent des vocabulaires différents. Deuxièmement, elle réduit l'ambiguïté : une compétence peut être rattachée à une famille, à un métier ou à un niveau d'exigence. Troisièmement, elle rend possible l'explicabilité : le système peut justifier une recommandation en indiquant quelles compétences sont communes, quelles compétences manquent et quelles formations permettent de combler l'écart.

Cependant, une nomenclature tabulaire reste insuffisante si elle ne représente que des listes. Le marché de l'emploi est relationnel par nature : un métier exige des compétences, une compétence appartient à une famille, une formation développe certaines compétences, une compétence peut être proche ou complémentaire d'une autre, et un candidat peut viser plusieurs trajectoires professionnelles. La modélisation en graphe devient alors pertinente, car elle permet de représenter explicitement ces relations. Dans un graphe de connaissances, les entités *Candidat*, *Offre*, *Métier*, *Compétence*, *Formation*, *Secteur* et *Niveau* sont reliées par des arcs sémantiques tels que « possède », « exige », « développe », « est proche de » ou « appartient à ».

Ce passage de la nomenclature vers le graphe est essentiel pour le présent mémoire. La nomenclature apporte le vocabulaire contrôlé ; le graphe apporte la structure relationnelle ; les embeddings et les LLMs apportent la capacité à traiter les formulations non structurées. L'enjeu méthodologique consiste donc à articuler ces trois niveaux : normaliser les métiers et compétences, représenter leurs relations dans Neo4j, puis exploiter ces relations pour calculer un *skill gap* et recommander des formations adaptées. Sans cette couche de standardisation, le système risque de produire des correspondances superficielles ; sans la couche graphe, il risque de perdre l'information relationnelle nécessaire à une recommandation explicable.

1.2 Les fondements des LLMs

L'avènement des LLMs a profondément redéfini le champ de l'IA appliquée au langage. **Alammar et Grootendorst** (2024) qualifient l'année 2023 d'« année de l'IA générative », soulignant qu'en l'espace de quelques mois, ces modèles ont transformé en profondeur des tâches aussi variées que la traduction automatique, la génération de texte, la synthèse documentaire et la recommandation. Pour comprendre cette rupture technologique, il est nécessaire d'en restituer les fondements théoriques. Cette section retrace d'abord l'évolution historique du NLP jusqu'aux LLMs, présente ensuite l'architecture Transformer et le mécanisme d'attention qui en constitue le cœur, explicite la notion d'embedding et de représentation vectorielle, avant de comparer les principales variantes architecturales des LLMs.

1.2.1 Définition et évolution historique des LLMs en NLP

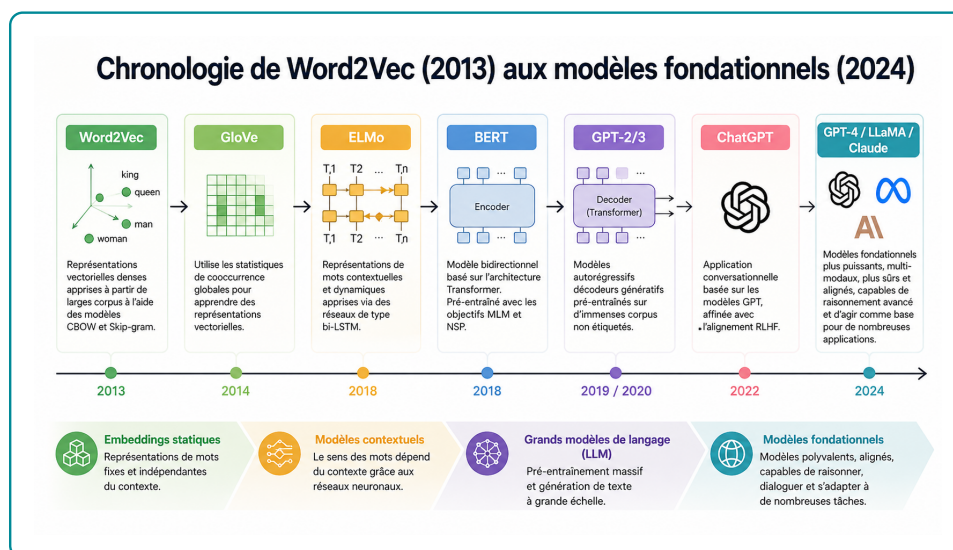
Le NLP, désigne le sous-domaine de l'IA dédié au développement de technologies capables de comprendre, traiter et générer le langage humain (Alammar & Grootendorst, 2024, chap. 1). Les deux termes sont aujourd'hui employés de manière largement interchangeable, en raison du succès continu des méthodes d'apprentissage automatique dans le traitement des problèmes linguistiques.

Un modèle de langage est, dans sa définition la plus générale, un système probabiliste qui attribue une probabilité à une séquence de mots (ou plus précisément à une séquence de tokens). Un Large Language Model est un modèle de langage à très grand nombre de paramètres généralement entraîné sur un corpus textuel massif capable de représenter ou de générer du texte avec une cohérence proche de celle de l'humain. Alammar et Grootendorst (2024) insistent toutefois sur le caractère évolutif de cette définition : le qualificatif « large » est arbitraire et dépend de l'état de l'art à un instant donné. Les auteurs proposent une lecture étendue qui inclut, sous le vocable LLM, à la fois les modèles génératifs (de type GPT, qui produisent du texte) et les modèles de représentation (de type BERT, qui produisent des plongements vectoriels exploitables pour la classification, la recherche sémantique ou le clustering). L'histoire du NLP peut être structurée en quatre grandes étapes successives comme illustre la figure 1.1 :

1.2.1.1 Les approches statistiques et le sac-de-mots (Bag-of-Words)

La représentation textuelle par sac-de-mots, théorisée dans les années 1950 et popularisée dans les années 2000, constitue le point de départ historique du NLP moderne. Elle consiste à découper un texte en tokens (généralement des mots), à construire un vocabulaire, puis à représenter chaque document par un vecteur de comptages. Bien que simple et toujours utile en complément de modèles plus récents, cette approche présente une limite fondamentale : elle ignore la sémantique et le contexte, considérant le texte comme un simple ensemble non ordonné de mots.

FIGURE 1.1 – Évolution chronologique des LLMs (2013-2024)



Source : Auteur

1.2.1.2 Les représentations vectorielles denses Word2Vec (2013) :

L'introduction de Word2Vec par Mikolov et al. (2013) marque une rupture majeure. A l'aide d'un réseau de neurones peu profond, le modèle apprend des plongements vectoriels (embeddings) qui capturent les régularités sémantiques entre mots, en s'appuyant sur l'hypothèse distributionnelle selon laquelle « des mots qui apparaissent dans des contextes similaires ont des sens similaires ». Pour la première fois, des opérations algébriques sur les vecteurs de mots (par exemple roi - homme + femme = reine) deviennent significatives. Word2Vec produit toutefois des représentations statiques : le mot « banque » a la même représentation, qu'il désigne une institution financière ou la rive d'un fleuve.

1.2.1.3 Les réseaux récurrents et l'introduction de l'attention (2014-2016)

Pour modéliser le caractère séquentiel du langage, les RNN, notamment dans leurs variantes LSTM et GRU, ont été largement adoptés. Couplés en architecture encodeur-décodeur, ils ont permis d'aborder des tâches complexes comme la traduction automatique. Cependant, le traitement séquentiel impose la compression de toute l'information d'une phrase d'entrée dans un unique vecteur de contexte, ce qui dégrade les performances sur les longues séquences. Bahdanau, Cho et Bengio (2014) introduisent alors le mécanisme d'attention, qui permet au décodeur de pondérer dynamiquement les états cachés de l'encodeur et de « se concentrer » sur les portions pertinentes de la séquence d'entrée à chaque étape de génération.

1.2.1.4 Le Transformer et l'ère des LLMs (2017-aujourd'hui)

En 2017, Vaswani et al. publient l'article fondateur « Attention Is All You Need », qui propose une architecture entièrement fondée sur l'attention et abandonne la récurrence. L'architecture Transformer se révèle hautement parallélisable, ce qui ouvre la voie à un passage à l'échelle sans précédent. Elle donne naissance, dès 2018, à deux familles de modèles :

- **BERT** (Bidirectional Encoder Representations from Transformers, Devlin et al., 2018), modèle encodeur seul, dédié à la représentation contextuelle du texte ;
- **GPT** (Generative Pre-trained Transformer, Radford et al., 2018), modèle décodeur seul, dédié à la génération de texte.

La trajectoire des modèles GPT illustre la dynamique d'échelle qui caractérise l'ère des LLMs : 117 millions de paramètres pour GPT-1 (2018), 1,5 milliard pour GPT-2 (2019), 175 milliards pour GPT-3 (2020). Le lancement public de ChatGPT en novembre 2022, fondé sur GPT-3.5 puis GPT-4, marque la diffusion massive de ces technologies, atteignant 100 millions d'utilisateurs actifs en deux mois. Parallèlement, l'écosystème open source (LLaMA, Mistral, Falcon) et de nouvelles architectures (Mamba, RWKV) viennent enrichir le paysage en proposant des compromis variés entre coût computationnel, longueur de contexte et qualité de génération.

1.2.2 Le concept d'embeddings et représentations vectorielles

Le langage humain est une donnée non structurée que les ordinateurs ne peuvent traiter directement sous sa forme brute. Pour qu'un modèle de langage puisse manipuler et « calculer » le sens, il est indispensable de convertir au préalable le texte en représentations numériques appelées vecteurs. Cette opération de transformation porte le nom d'embedding (plongement vectoriel). Les *embeddings* constituent la représentation numérique fondamentale sur laquelle repose tout le pipeline de matching emploi-compétences. Un embedding est une projection d'un objet (mot, phrase, document, compétence) dans un espace vecto-

riel continu de dimension d , typiquement $d \in [128, 4096]$, où la proximité géométrique reflète la similarité sémantique. L'objectif d'un modèle d'embedding est de produire des représentations qui restituent, dans un espace mathématique, le sens des unités linguistiques de la manière la plus précise possible. Autrement dit, l'embedding constitue le pont entre le langage humain discret, symbolique et fortement ambigu et les opérations algébriques que les réseaux de neurones sont en mesure d'effectuer.

1.2.2.1 Le rôle des embeddings dans l'architecture Transformer

Dans un modèle Transformer, les embeddings constituent la première étape du traitement. Le pipeline d'entrée se déroule selon les étapes suivantes :

1. Le texte d'entrée est segmenté en jetons (tokens) par un algorithme de tokenisation, généralement de type sous-mots (Byte-Pair Encoding, WordPiece ou SentencePiece).
2. Chaque jeton est associé à un vecteur initial issu d'une matrice d'embeddings apprise lors du pré-entraînement et indexée par les identifiants des jetons du vocabulaire.
3. Un codage positionnel (positional encoding) est ajouté à ces vecteurs afin que le modèle puisse exploiter l'ordre des mots dans la séquence, l'architecture Transformer traitant l'ensemble des jetons simultanément et n'ayant donc, par construction, aucune notion intrinsèque d'ordre.
4. Les vecteurs ainsi enrichis circulent à travers les couches successives du modèle, où ils sont progressivement contextualisés par les mécanismes d'auto-attention et les réseaux de propagation avant.

1.2.3 L'architecture Transformer et le mécanisme d'attention

L'architecture Transformer, introduite par VASWANI et al. (2017) dans le célèbre article *Attention Is All You Need*, constitue le paradigme architectural dominant dans le traitement du langage naturel. Elle repose sur un mécanisme d'auto-attention (*self-attention*) qui permet au modèle d'examiner simultanément l'ensemble des positions d'une séquence d'entrée pour calculer des représentations contextualisées de chaque token.

1.2.3.1 Le mécanisme d'auto-attention

Formellement, étant donnée une séquence de vecteurs d'entrée $\mathbf{X} \in \mathbb{R}^{n \times d}$, le mécanisme d'attention produit une sortie en calculant trois matrices : les requêtes $\mathbf{Q}$, les clés $\mathbf{K}$ et les valeurs $\mathbf{V}$, obtenues par projections linéaires apprises :

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V \quad (1.1)$$

La sortie de l'attention est alors :

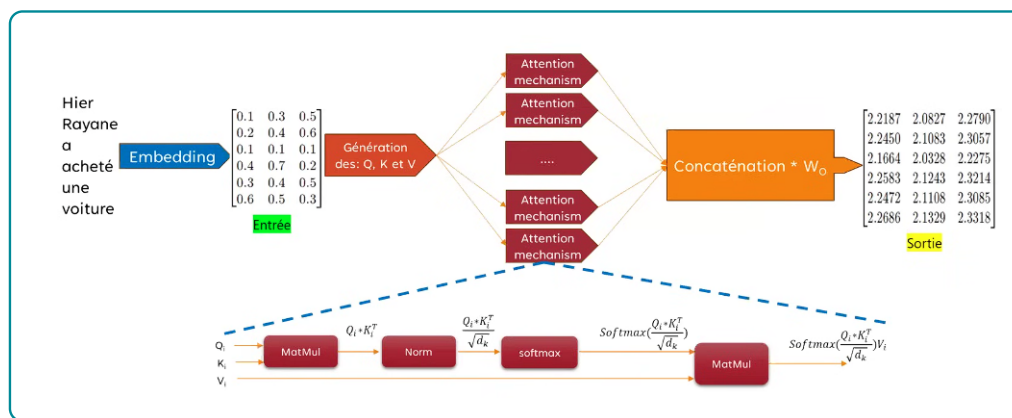
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (1.2)$$

où d_k est la dimension des vecteurs clés, utilisée pour normaliser le produit scalaire et éviter des gradients trop faibles. Ce mécanisme permet à chaque token de « regarder » tous les autres tokens de la séquence simultanément, contrairement aux architectures récurrentes (RNN, LSTM) qui traitent les tokens séquentiellement. Dans le contexte du matching emploi-compétences, cela signifie que le modèle peut mettre en relation le terme « Python » d'un CV avec « développement back-end » dans une offre d'emploi, même si ces termes sont distants dans le texte.

1.2.3.2 L'attention multi-têtes

L'architecture utilise Afin de capturer simultanément des relations linguistiques de natures variées, le Transformer ne se limite pas à un unique calcul d'attention. Il met en œuvre une attention multi-têtes, c'est-à-dire plusieurs calculs d'attention exécutés en parallèle (typiquement 8 ou 12 têtes, parfois davantage dans les modèles de plus grande taille). Chaque tête d'attention dispose de ses propres matrices de projection Q, K et V , apprises de manière indépendante comme le montre la 1.2. Elle peut ainsi se spécialiser dans l'identification d'un type particulier de relation : dépendances syntaxiques, proximité sémantique, coréférence pronominale, reconnaissance d'entités, etc. Les sorties des différentes têtes sont ensuite concaténées puis projetées linéairement pour produire la représentation finale. Cette architecture confère au modèle une capacité d'expression supérieure et lui permet d'analyser une même séquence sous plusieurs angles complémentaires.

FIGURE 1.2 – Mécanisme de self-attention



Source : Auteur

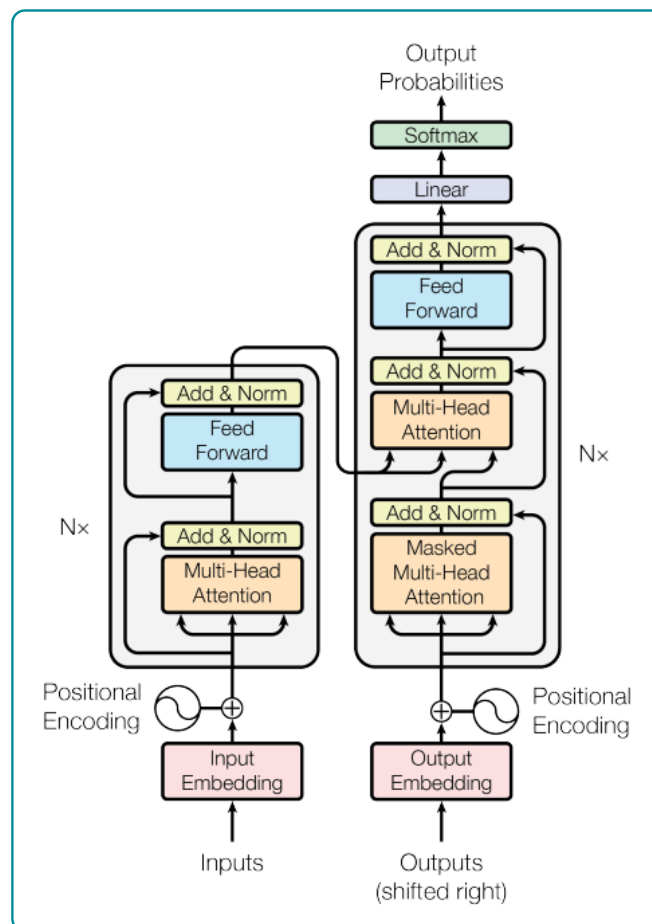
1.2.3.3 Composants internes d'un bloc Transformer

Au-delà de l'attention, la figure 1.3 montre que chaque bloc Transformer comprend deux éléments complémentaires essentiels au bon fonctionnement de l'architecture :

- **Le réseau de neurones à propagation avant (Feedforward Neural Network, FFN).** Appliqué de manière indépendante à chaque position de la séquence, il constitue la majeure partie de la capacité de traitement et de mémorisation du modèle. C'est dans cette sous-couche qu'est essentiellement stockée la connaissance acquise lors du pré-entraînement.
- **Les connexions résiduelles et la normalisation de couche.** Les connexions résiduelles (residual connections) facilitent la propagation de l'information et du gradient à travers les couches profondes, tandis que les techniques de normalisation Layer Normalization dans la formulation originale, RMSNorm dans les architectures plus récentes stabilisent l'entraînement et accélèrent la convergence.

L'enchaînement de ces éléments self-attention, normalisation, FFN, connexions résiduelles est répété un grand nombre de fois (de 12 couches pour les modèles de petite taille à plus d'une centaine pour les plus grands LLMs), ce qui permet au modèle de construire des représentations linguistiques de plus en plus abstraites au fil des couches.

FIGURE 1.3 – Architecture complète du Transformer



Source : Auteur

1.3 Généralité sur le fine-tuning des modèles de LLMs

Le *fine-tuning*, ou ajustement fin, désigne l'adaptation d'un modèle pré-entraîné à une tâche, un domaine ou un style de réponse spécifique. Dans le cas des LLMs, le modèle dispose déjà de connaissances linguistiques et statistiques acquises lors d'un pré-entraînement massif sur de grands corpus. Le fine-tuning ne repart donc pas de zéro : il modifie une partie ou la totalité des paramètres du modèle afin d'améliorer ses performances sur un corpus plus spécialisé [9, 10]. Cette logique est particulièrement pertinente lorsque le domaine cible possède un vocabulaire, des relations sémantiques ou des critères de décision qui diffèrent des usages généraux du langage.

Dans le cadre du matching emploi-compétences, l'intérêt du fine-tuning est direct. Les textes manipulés ne sont pas de simples phrases ordinaires ; ce sont des CV, des offres d'emploi, des intitulés de métiers, des descriptions de tâches, des compétences techniques, des certifications et des parcours de formation. Un modèle généraliste peut comprendre une partie de ces informations, mais il peut confondre des compétences proches, ignorer des synonymes professionnels locaux ou mal hiérarchiser l'importance d'une compétence dans une offre. L'ajustement sur des données du domaine permet de spécialiser les représentations du modèle afin qu'il rapproche mieux les profils et les postes réellement compatibles.

1.3.1 Pré-entraînement, adaptation et spécialisation

Le pré-entraînement consiste à exposer un modèle à de très grands volumes de textes afin qu'il apprenne des régularités générales du langage. Selon l'architecture, l'objectif peut être de prédire des mots masqués, comme dans BERT, ou de prédire le prochain token, comme dans les modèles génératifs de type GPT [9, 11]. Cette phase produit un modèle généraliste, puissant mais non spécialisé.

Le fine-tuning intervient ensuite comme une phase d'adaptation. On fournit au modèle des exemples représentatifs du problème cible : paires CV-offre, couples compétence-métier, descriptions de formation-compétences développées, ou encore triplets du type *offre, compétence requise, niveau attendu*. Le modèle apprend alors à ajuster ses représentations internes aux régularités du domaine. Dans un système emploi-compétences, l'objectif n'est pas seulement de générer du texte ; il est surtout d'améliorer l'extraction d'entités, la classification des compétences, la recherche sémantique et le classement des recommandations.

Il faut néanmoins éviter une erreur fréquente : fine-tuner un LLM ne garantit pas automatiquement un meilleur système. Si les données sont faibles, bruitées ou biaisées, le modèle apprendra ces biais. Si les labels sont incohérents, le fine-tuning dégradera la qualité des représentations. Si le domaine est mal défini, l'adaptation risque d'être superficielle. La qualité du corpus, la définition des tâches et la cohérence des annotations sont donc plus importantes que la seule taille du modèle.

1.3.2 Les principales formes de fine-tuning

On distingue plusieurs formes d'adaptation, selon l'objectif et les ressources disponibles.

1.3.2.1 Fine-tuning supervisé

Le fine-tuning supervisé utilise des données annotées. Dans le cas du matching emploi-compétences, il peut s'agir de paires *profil-offre* annotées comme pertinentes ou non pertinentes, de compétences extraites manuellement de CV, ou de descriptions associées à une catégorie métier. Le modèle apprend à reproduire ces décisions. Cette approche est utile pour des tâches comme la classification, l'extraction d'entités et le scoring de similarité.

1.3.2.2 Instruction tuning

L'*instruction tuning* adapte le modèle à suivre des consignes formulées en langage naturel. Au lieu de seulement prédire une classe ou une similarité, le modèle apprend à répondre à des demandes telles que : « extrais les compétences techniques de ce CV », « identifie les compétences manquantes pour cette offre » ou « propose une formation pour combler cet écart ». Les travaux d'Ouyang et al. montrent que l'ajustement sur des instructions et préférences humaines améliore l'utilité perçue des réponses des grands modèles de langage [12].

1.3.2.3 Fine-tuning contrastif pour les embeddings

Pour la recherche sémantique, le fine-tuning peut viser non pas la génération de texte, mais la qualité des embeddings. L'objectif est alors de rapprocher dans l'espace vectoriel les éléments similaires et d'éloigner les éléments non similaires. Par exemple, une offre de « data analyst » et un profil maîtrisant Python, SQL, Excel, visualisation et statistiques doivent être proches ; une offre de comptabilité pure et un profil de développement web doivent être plus éloignés. Cette logique de fine-tuning contrastif est centrale pour les modèles de type Sentence-BERT [13].

1.3.2.4 Adaptation légère des paramètres

Le fine-tuning complet d'un LLM peut être coûteux, car il met à jour tous les paramètres du modèle. Les méthodes d'adaptation efficace, comme LoRA, réduisent ce coût en insérant de petites matrices entraînables dans certaines couches du réseau, tout en gardant la majorité des paramètres gelés [14]. QLoRA pousse cette logique plus loin en combinant quantification et adaptation légère, ce qui permet d'ajuster de grands modèles avec beaucoup moins de mémoire GPU [15]. Pour un projet appliqué comme celui-ci, ces méthodes sont souvent plus réalistes qu'un fine-tuning complet.

1.3.3 Application au matching emploi-compétences

Dans le système proposé, le fine-tuning peut intervenir à trois niveaux complémentaires. Le premier niveau concerne l'extraction : le modèle doit identifier dans les textes les compétences, outils, diplômes, années d'expérience, secteurs et métiers. Le deuxième niveau concerne la représentation : les embeddings doivent capturer les similarités professionnelles, y compris lorsque les formulations sont différentes. Le troisième niveau concerne l'explication : le modèle doit être capable de produire une justification lisible, par exemple en distinguant les compétences acquises, les compétences manquantes et les formations recommandées.

Cette architecture suppose une articulation rigoureuse avec la nomenclature métiers-compétences. Le fine-tuning ne doit pas seulement apprendre des régularités textuelles ; il doit être contraint par un référentiel. Par exemple, si le modèle extrait « Power BI », « tableau de bord » et « reporting », ces éléments doivent pouvoir être rattachés à une compétence standardisée de visualisation ou d'aide à la décision. C'est cette normalisation qui permet ensuite d'alimenter correctement le graphe de connaissances.

1.3.4 Risques et précautions méthodologiques

Le fine-tuning présente plusieurs risques. Le premier est le sur-apprentissage : le modèle peut mémoriser les exemples du corpus sans généraliser à de nouvelles offres. Le deuxième est le biais de données : si les offres collectées favorisent certains secteurs, niveaux de diplôme ou formulations linguistiques, le modèle reproduira ces déséquilibres. Le troisième est la perte de robustesse : un modèle trop spécialisé peut devenir moins performant sur des formulations inattendues. Enfin, le quatrième risque est l'opacité : si le modèle produit un score sans explication, il sera difficile de convaincre un utilisateur ou un recruteur de la pertinence de la recommandation.

Ces limites justifient le recours à un système hybride. Le LLM apporte la compréhension linguistique ; la base vectorielle apporte la recherche par similarité ; le graphe de connaissances apporte la structure, les contraintes et l'explicabilité. Dans cette perspective, le fine-tuning n'est pas une fin en soi. Il est un levier d'adaptation qui doit rester subordonné à l'objectif central du mémoire : produire un matching emploi-compétences plus pertinent, plus interprétable et plus utile pour la montée en compétences.

1.4 Généralité sur des BD vectorielles et orientées graphes

Les bases de données vectorielles et les bases orientées graphes répondent à deux besoins complémentaires dans les systèmes modernes d'intelligence artificielle. Les premières sont conçues pour indexer et interroger des représentations numériques continues, généralement produites par des modèles d'embeddings. Les secondes sont conçues pour représenter explicitement des entités et les relations qui les relient.

Dans le cadre du matching emploi-compétences, cette complémentarité est stratégique : la base vectorielle permet de retrouver des profils et offres proches sur le plan sémantique, tandis que le graphe permet d'expliquer pourquoi ces éléments sont liés à travers des métiers, compétences, secteurs, formations et niveaux d'exigence.

1.4.1 Structure générale d'une base de données vectorielle

Une base de données vectorielle est un système de stockage et de recherche optimisé pour des vecteurs numériques de grande dimension. Chaque objet textuel, par exemple un CV, une offre d'emploi, une compétence ou une description de formation, est transformé en embedding par un modèle de représentation. Cet embedding est ensuite stocké avec un identifiant, des métadonnées et parfois le texte source. La base ne manipule donc pas seulement des chaînes de caractères ; elle manipule des points dans un espace vectoriel où la distance ou la proximité traduit une forme de similarité sémantique.

La structure minimale d'une base vectorielle comprend généralement quatre composantes. La première est le *document source*, c'est-à-dire l'information initiale : CV, offre, compétence, métier ou contenu de formation. La deuxième est le *vecteur d'embedding*, qui encode ce document dans un espace numérique. La troisième est l'ensemble des *métadonnées*, comme le pays, le secteur, le niveau d'expérience, la date de collecte ou le type d'objet. La quatrième est l'*index vectoriel*, qui accélère la recherche des plus proches voisins.

Dans un système de matching emploi-compétences, une table vectorielle peut par exemple contenir les colonnes suivantes : un identifiant d'offre, le texte nettoyé de l'offre, le vecteur associé, le secteur d'activité, le niveau d'expérience requis et la liste normalisée des compétences extraites. Une requête utilisateur, comme un profil candidat, est vectorisée à son tour. Le système compare ensuite ce vecteur avec ceux de la base afin d'identifier les offres les plus proches.

La difficulté principale vient du fait que les embeddings sont souvent de grande dimension. Une recherche exhaustive sur tous les vecteurs devient coûteuse lorsque la base contient des milliers ou millions d'objets. C'est pourquoi les bases vectorielles utilisent des index de recherche approximative des plus proches voisins, tels que HNSW ou IVF, largement employés dans les moteurs spécialisés comme FAISS ou dans des extensions comme pgvector [16, 17, 18].

1.4.2 Principe de la recherche sémantique dans une BD vectorielle

La recherche sémantique consiste à retrouver des objets proches par le sens plutôt que par la présence exacte des mêmes mots. Contrairement à une recherche lexicale classique, qui dépend fortement des mots-clés, elle exploite la position des embeddings dans un espace vectoriel. Deux textes peuvent donc être rapprochés même s'ils n'emploient pas les mêmes termes. Par exemple, une offre mentionnant « conception de tableaux de bord décisionnels » peut être rapprochée d'un profil indiquant « Power BI, reporting et visualisation de données ».

Le processus comprend généralement cinq étapes. Premièrement, les textes sont nettoyés et segmentés. Deuxièmement, un modèle d'embedding transforme chaque texte en vecteur. Troisièmement, les vecteurs sont stockés et indexés. Quatrièmement, la requête est elle-même transformée en vecteur. Cinquièmement, la base retourne les objets les plus proches selon une mesure de similarité.

Les mesures les plus utilisées sont la similarité cosinus, la distance euclidienne et le produit scalaire.

La similarité cosinus est particulièrement fréquente en traitement du langage, car elle mesure l'angle entre deux vecteurs et neutralise en partie l'effet de leur norme. Elle est définie par :

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (1.3)$$

Dans le cas du matching emploi-compétences, cette recherche permet de produire une première liste d'offres candidates pour un profil donné. Toutefois, un score vectoriel seul ne suffit pas. Il peut identifier une proximité globale, mais il ne dit pas toujours quelles compétences expliquent cette proximité, quelles compétences manquent, ni quelle formation pourrait combler l'écart. C'est précisément la limite qui justifie l'association avec une base orientée graphe.

1.4.3 Structure générale d'une base de données orientée graphe

Une base de données orientée graphe représente l'information sous forme de nœuds et de relations. Les nœuds désignent les entités du domaine, tandis que les relations décrivent les liens entre ces entités. Cette logique diffère des bases relationnelles classiques, où les relations sont généralement reconstituées par des jointures entre tables. Dans une base graphe, les liens sont des objets de première classe : ils sont stockés, typés, parcourus et analysés directement.

Dans le domaine emploi-compétences, les nœuds peuvent représenter des candidats, offres, métiers, compétences, secteurs, formations, certifications ou niveaux de qualification. Les relations peuvent exprimer des liens tels que *possède*, *exige*, *appartient à*, *développe*, *est similaire à* ou *prépare à*. Cette représentation est beaucoup plus proche de la structure réelle du marché de l'emploi, car un métier n'est pas une simple ligne de table : il est défini par un ensemble de compétences, d'activités, de niveaux, de secteurs et de trajectoires possibles.

Formellement, un graphe peut être défini comme un couple $G = (V, E)$, où V désigne l'ensemble des nœuds et E l'ensemble des relations. Dans un graphe de connaissances, les relations sont souvent représentées sous forme de triplets :

$$(\text{sujet}, \text{relation}, \text{objet}) \quad (1.4)$$

Par exemple, le triplet (Data Analyst, exige, SQL) signifie que la compétence SQL est requise pour le métier de data analyst. De même, (Formation Power BI, développe, Visualisation de données) indique qu'une formation contribue à développer une compétence standardisée.

Les bases graphes comme Neo4j permettent ensuite d'interroger ces relations avec des langages adaptés, notamment Cypher. Un système peut ainsi répondre à des questions que la recherche vectorielle seule gère mal : quelles compétences manquent à un candidat pour viser un métier ? Quelles formations développent ces compétences ? Quelles offres sont proches d'un profil si l'on accepte une trajectoire de formation courte ?

1.4.4 Techniques de description et d'analyse des graphes de connaissances

Un graphe de connaissances ne sert pas seulement à stocker des relations ; il permet aussi de les analyser. Les techniques de description de graphe visent à comprendre la structure du réseau, l'importance relative des entités et les chemins qui relient les objets. Dans un système emploi-compétences, ces analyses

peuvent aider à identifier les compétences centrales, les métiers passerelles, les formations les plus utiles ou les trajectoires de reconversion les plus plausibles.

Une première famille de techniques concerne les mesures de centralité. La centralité de degré mesure le nombre de connexions directes d'un nœud. Une compétence fortement connectée à de nombreux métiers, comme Excel, SQL ou communication professionnelle, peut être considérée comme transversale. La centralité d'intermédiarité mesure la fréquence avec laquelle un nœud se trouve sur les chemins les plus courts entre d'autres nœuds ; elle permet d'identifier des compétences ou formations jouant un rôle de passerelle entre plusieurs familles de métiers. La centralité de proximité mesure la distance moyenne d'un nœud à tous les autres, indiquant sa capacité à atteindre rapidement le reste du réseau.

Une deuxième famille concerne la recherche de chemins. Dans le matching emploi-compétences, les chemins permettent de produire des recommandations explicables. Si un candidat possède Python et statistiques, et qu'une offre exige aussi SQL et Power BI, le graphe peut identifier les compétences manquantes, puis rechercher les formations qui les développent. L'explication ne repose donc pas uniquement sur un score opaque ; elle repose sur un chemin interprétable entre profil, compétences, offre et formation.

Une troisième famille concerne la détection de communautés. Elle permet de repérer des groupes de métiers ou de compétences fortement liés entre eux. Par exemple, les métiers de la data peuvent former une communauté autour des compétences Python, SQL, statistiques, visualisation et machine learning, tandis que les métiers du marketing numérique peuvent former une autre communauté autour du SEO, de la publicité en ligne, de l'analyse d'audience et de la stratégie de contenu. Ces regroupements peuvent aider à structurer les parcours de formation et les recommandations de reconversion.

Enfin, les graphes de connaissances peuvent être enrichis par des représentations vectorielles de graphes, ou *graph embeddings*. Ces méthodes projettent les nœuds et relations dans un espace vectoriel afin de faciliter la prédiction de liens, la classification de nœuds ou la recommandation. Des approches plus avancées, comme les réseaux de neurones sur graphes, exploitent la structure locale du graphe pour apprendre des représentations tenant compte du voisinage des entités [19, 20].

1.4.5 Synergie LLM + Graphes de Connaissances

La combinaison entre LLMs et graphes de connaissances répond à une faiblesse de chacun des deux outils pris isolément. Les LLMs sont puissants pour comprendre et générer du langage naturel, extraire des entités et reformuler des contenus. Cependant, ils peuvent produire des réponses approximatives, manquer de traçabilité ou halluciner des relations non vérifiées. Les graphes de connaissances, à l'inverse, offrent une structure explicite, contrôlable et interrogeable, mais ils nécessitent une alimentation régulière et une normalisation rigoureuse des données.

Dans un pipeline emploi-compétences, les LLMs peuvent intervenir en amont pour extraire automatiquement les entités présentes dans les CV et offres d'emploi : métiers, compétences, diplômes, outils, années d'expérience, secteurs et certifications. Ces entités sont ensuite alignées sur une nomenclature standardisée, comme la NCM ou ESCO, puis insérées dans le graphe sous forme de nœuds et relations. Le graphe sert alors de mémoire structurée du système.

La synergie apparaît aussi au moment de la recommandation. Une base vectorielle peut identifier les offres les plus proches d'un profil, mais le graphe permet de vérifier la cohérence métier de cette proximité. Par exemple, si deux offres sont proches vectoriellement mais appartiennent à des familles

de métiers différentes, le graphe peut aider à distinguer une proximité linguistique superficielle d'une proximité professionnelle réelle. Inversement, le graphe peut révéler des correspondances indirectes que la recherche vectorielle ne voit pas, comme une compétence transférable entre deux métiers voisins.

Enfin, cette combinaison améliore l'explicabilité. Un LLM peut générer une recommandation lisible en langage naturel, mais cette recommandation doit être fondée sur des relations vérifiables dans le graphe : compétences possédées, compétences exigées, compétences manquantes, formations associées et trajectoire possible. Le rôle du LLM n'est donc pas d'inventer l'explication ; il est de verbaliser une chaîne de raisonnement structurée par le graphe. Cette logique rejoint les approches de RAG et de GraphRAG, qui cherchent à combiner récupération d'information, génération et structure relationnelle pour produire des réponses plus contextualisées et plus contrôlables [21, 19].

1.5 Généralité sur les systèmes de recommandation

Les systèmes de recommandation (SR) sont des outils algorithmiques conçus pour prédire et suggérer des éléments pertinents à un utilisateur, en se basant sur des informations le concernant ainsi que sur les caractéristiques des éléments disponibles. Appliqués au marché de l'emploi, ils permettent de mettre en relation candidats et offres de manière automatisée, personnalisée et à grande échelle. Dans un système classique de recommandation, on distingue généralement trois types d'entités : les utilisateurs, les items et les interactions. Dans le commerce électronique, l'utilisateur est un client et l'item est un produit. Dans le cas du présent mémoire, l'utilisateur peut être un candidat ou un membre de la communauté KmerAI, tandis que l'item peut être une offre d'emploi, un métier cible, une compétence ou une formation. Les interactions peuvent correspondre à des candidatures, clics, sauvegardes d'offres, évaluations, historiques de formation ou correspondances validées par un expert.

La recommandation appliquée à l'emploi présente cependant une difficulté plus forte que la recommandation de produits. Une mauvaise recommandation de film a un coût faible ; une mauvaise recommandation professionnelle peut orienter un candidat vers une trajectoire peu adaptée. Le système doit donc prendre en compte la pertinence, mais aussi l'explicabilité, l'équité, le niveau réel du candidat et la faisabilité de la montée en compétences.

1.5.1 Filtrage basé sur le contenu (*Content-Based Filtering*)

Le filtrage basé sur le contenu recommande des items similaires à ceux qui correspondent déjà au profil ou aux préférences de l'utilisateur. Il repose sur les caractéristiques des objets. Dans le cas du matching emploi-compétences, ces caractéristiques peuvent être les compétences exigées, le secteur, le niveau d'expérience, le type de contrat, le lieu, le domaine de formation ou les outils techniques demandés.

Son principe est simple : si un candidat possède Python, SQL, statistiques et visualisation de données, le système recherchera des offres contenant des caractéristiques proches, par exemple analyste de données, chargé d'études statistiques ou business intelligence analyst. Les embeddings permettent d'améliorer cette approche en dépassant les mots-clés exacts. Le système peut alors rapprocher « reporting décisionnel » de « tableau de bord » ou « visualisation de données ».

L'avantage principal du filtrage basé sur le contenu est son interprétabilité relative : on peut expliquer une recommandation par les caractéristiques communes entre le profil et l'offre. Sa limite est qu'il tend à enfermer l'utilisateur dans ce qu'il possède déjà. Si un candidat a un profil statistique, le système peut

négliger des trajectoires proches mais exigeant une courte montée en compétences vers la data engineering ou le marketing analytics. Cette limite justifie l'introduction du skill gap et des relations de proximité dans le graphe [22].

1.5.2 Des méthodes traditionnelles aux modèles séquentiels et neuronaux

La revue récente de Redjda sur les modèles de langage appliqués à la recommandation basée sur les sessions fournit une grille de lecture utile pour situer l'évolution des systèmes de recommandation [23]. Même si son terrain porte sur la prédiction du prochain item dans des sessions d'interaction, la logique générale est transposable au matching emploi-compétences : les méthodes traditionnelles sont efficaces lorsque les signaux sont simples, stables et bien observés, mais elles deviennent limitées dès que la décision dépend d'une séquence, d'un contexte, d'une représentation sémantique ou de relations complexes entre entités.

Les approches classiques peuvent être regroupées en trois familles. Les méthodes par règles et mots-clés rapprochent des profils et des offres à partir de termes communs. Elles sont simples à expliquer, mais elles échouent lorsque les mêmes compétences sont décrites avec des formulations différentes. Les méthodes collaboratives exploitent les comportements passés des utilisateurs ou des candidats : consultation, candidature, validation, recrutement ou suivi de formation. Elles peuvent capter des régularités collectives, mais elles exigent un historique fiable et souffrent fortement du démarrage à froid [24]. Les méthodes séquentielles, enfin, modélisent l'ordre des interactions afin de prédire l'action suivante ; elles sont centrales dans la recommandation basée sur les sessions, comme le montrent les travaux sur les RNN et les mécanismes d'attention [25, 26].

Dans le cas du marché de l'emploi camerounais, ces trois familles ne suffisent pas prises isolément. Les données d'interaction candidat-offre sont souvent rares ou incomplètes ; les intitulés de postes sont hétérogènes ; les compétences peuvent être implicites dans les descriptions ; et la décision ne dépend pas uniquement de la préférence du candidat, mais aussi de la faisabilité professionnelle. Une offre pertinente doit être proche du profil, compatible avec le niveau de formation, cohérente avec le secteur visé et accessible après une montée en compétences raisonnable. Le problème n'est donc pas seulement de prédire un clic ou une candidature ; il est de produire une recommandation défendable pour un recruteur, un conseiller d'orientation ou un candidat.

1.5.3 Apports des graphes et des GNN dans la recommandation

Une contribution importante de la littérature récente est d'avoir montré l'intérêt des structures de graphe pour représenter les relations entre utilisateurs, items et contextes. Redjda rappelle que les approches à base de graphes se sont imposées dans la recommandation séquentielle parce qu'elles capturent mieux les transitions et les dépendances complexes entre items que les modèles purement linéaires [23]. Les travaux sur les graph neural networks et les modèles de recommandation fondés sur les graphes montrent ainsi que la performance ne vient pas seulement de la représentation individuelle des objets, mais aussi de l'information portée par leurs relations [27, 28].

Cette idée est directement pertinente pour l'emploi. Un candidat, une offre, un métier, une compétence, une formation et un secteur ne sont pas des objets indépendants. Ils forment un système relationnel : une offre exige des compétences, un candidat possède ou développe certaines compétences, une formation

prépare à un domaine, un niveau NCF contraint l'accès à un poste, et un groupe MEPC situe le métier dans une nomenclature nationale. Un graphe de connaissances permet donc de représenter explicitement ces liens au lieu de les laisser implicites dans des vecteurs ou des textes.

La différence entre un graphe de connaissances et un GNN doit cependant être clarifiée. Le présent mémoire ne prétend pas entraîner un GNN complet sur les relations emploi-compétences. Il mobilise d'abord le graphe comme structure explicative et interrogeable : Neo4j sert à vérifier des contraintes, à calculer des écarts de compétences et à tracer les chemins justifiant une recommandation. Cette position est plus prudente et plus défendable dans un contexte où le volume d'interactions validées reste limité. Les GNN constituent une extension possible lorsque les données relationnelles et les labels de validation seront suffisamment nombreux.

1.5.4 Transformers et modèles de langage pour la recommandation

L'autre apport central mis en évidence par Redjda est le rapprochement entre la recommandation séquentielle et les modèles de langage Transformer. La prédiction du prochain item dans une session peut être rapprochée de la prédiction du prochain token en NLP : dans les deux cas, le modèle apprend à exploiter un contexte pour anticiper l'élément suivant [29, 30, 23]. Cette analogie explique l'intérêt croissant pour BERT, GPT, DeBERTa et les architectures de type Transformer dans les systèmes de recommandation.

Pour le présent mémoire, l'enjeu n'est pas de prédire le prochain clic dans une session, mais de représenter correctement des textes d'offres et de profils. Le rôle des Transformers est donc différent : ils servent principalement à construire des embeddings sémantiques. Un encodeur comme Sentence-BERT transforme une phrase, une annonce ou un profil en vecteur dense, ce qui permet de mesurer la proximité de sens entre deux descriptions [13]. Cette approche est plus adaptée qu'un simple sac de mots lorsque les annonces contiennent des formulations variables, des synonymes, des termes français et anglais, ou des compétences formulées de manière indirecte.

La logique de fine-tuning utilisée dans ce projet s'inscrit dans cette continuité. Le modèle n'est pas entraîné à générer du texte ; il est adapté pour mieux rapprocher deux vues d'une même offre : une vue structurée, issue des métadonnées, et une vue textuelle, issue des compétences et détails de l'annonce. Cette formulation transforme le problème en tâche de recherche sémantique. Elle est cohérente avec la littérature sur les représentations de phrases, mais elle appelle une limite importante : un bon score vectoriel ne prouve pas une compatibilité professionnelle. Il ne dit pas encore si le niveau NCF est suffisant, si les compétences essentielles sont acquises, ni si la trajectoire de formation est réaliste.

1.5.5 Vers une recommandation hybride orientée recrutement et montée en compétences

La synthèse de la littérature conduit à une conclusion méthodologique forte : dans un système emploi-compétences, aucune famille de méthodes ne suffit seule. Le contenu textuel apporte la proximité sémantique ; les référentiels apportent la standardisation ; le graphe apporte la structure relationnelle ; les éventuels signaux collaboratifs apportent l'expérience d'usage lorsqu'ils existent ; et le module génératif apporte une explication en langage naturel. L'hybridation n'est donc pas un choix esthétique, mais une nécessité imposée par la nature du problème [31, 32].

Cette hybridation doit toutefois être disciplinée. Un score collaboratif ne doit pas être affirmé si l'historique d'interactions est insuffisant. Un modèle Transformer ne doit pas être présenté comme une preuve de compatibilité métier. Un graphe incomplet ne doit pas être interprété comme une vérité exhaustive sur les compétences. La recommandation robuste doit séparer les signaux : similarité sémantique, couverture des compétences, niveau de formation, cohérence sectorielle et possibilités de montée en compétences. C'est cette séparation qui permet de produire un verdict opérationnel : profil prêt à postuler, profil à recommander avec plan de formation, profil à conserver en vivier ou profil actuellement hors cible.

Ainsi, la revue de littérature oriente directement l'architecture retenue dans ce mémoire. Comme chez Redjidal, les Transformers sont mobilisés parce qu'ils apprennent des représentations riches utiles à la recommandation. Comme dans les approches à base de graphes, les relations entre objets sont indispensables pour dépasser une simple proximité vectorielle. Mais l'application au marché de l'emploi impose une contrainte supplémentaire : la recommandation doit être explicable, prudente et orientée vers la montée en compétences. Le système proposé articule donc recherche sémantique, graphe de connaissances et logique de skill gap afin de recommander non seulement une offre, mais aussi une trajectoire d'amélioration professionnelle.

1.6 Génération Augmentée par Récupération (RAG) et GraphRAG

La recherche sémantique consiste à représenter une requête et un ensemble de documents dans un même espace vectoriel, puis à classer les documents selon leur proximité avec la requête. Contrairement à une recherche par mots-clés, elle ne dépend pas uniquement de la présence des mêmes termes. Elle cherche plutôt à capturer la proximité de sens entre deux formulations. Dans le cadre de ce mémoire, cette propriété est essentielle : un profil mentionnant « analyse statistique, tableaux de bord et SQL » doit pouvoir être rapproché d'une offre de « data analyst », même lorsque les formulations exactes ne se recouvrent pas.

La RAG prolonge cette logique en combinant deux opérations : une étape de récupération d'information et une étape de génération. Le système commence par rechercher des passages, documents ou objets structurés pertinents dans une base externe ; il transmet ensuite ces éléments à un modèle génératif, qui produit une réponse conditionnée par le contexte récupéré [21]. Son fonctionnement peut être décomposé en cinq étapes. Premièrement, les documents sont préparés, découpés si nécessaire, puis encodés sous forme de vecteurs. Deuxièmement, ces vecteurs sont stockés dans un index ou une base vectorielle. Troisièmement, lorsqu'une requête arrive, elle est encodée dans le même espace vectoriel que les documents. Quatrièmement, le système récupère les éléments les plus proches selon une mesure comme la similarité cosinus. Cinquièmement, ces éléments sont ajoutés au prompt du modèle génératif afin qu'il produise une réponse fondée sur le contexte récupéré.

Dans cette architecture, le modèle de langage ne répond donc pas seulement à partir de ses connaissances internes. Il reçoit un contexte externe, généralement plus récent, plus spécifique ou plus local que ce qu'il a appris pendant son entraînement. L'intérêt est double. D'une part, le modèle génératif n'est plus obligé de s'appuyer uniquement sur ses paramètres internes, qui peuvent être incomplets ou obsolètes. D'autre part, la réponse peut être rattachée à des sources ou à des éléments vérifiables. Cette architecture reste toutefois fragile lorsque la récupération initiale est mauvaise : si le contexte fourni au modèle est incomplet, bruité ou mal classé, la génération finale peut devenir convaincante en surface mais faible sur

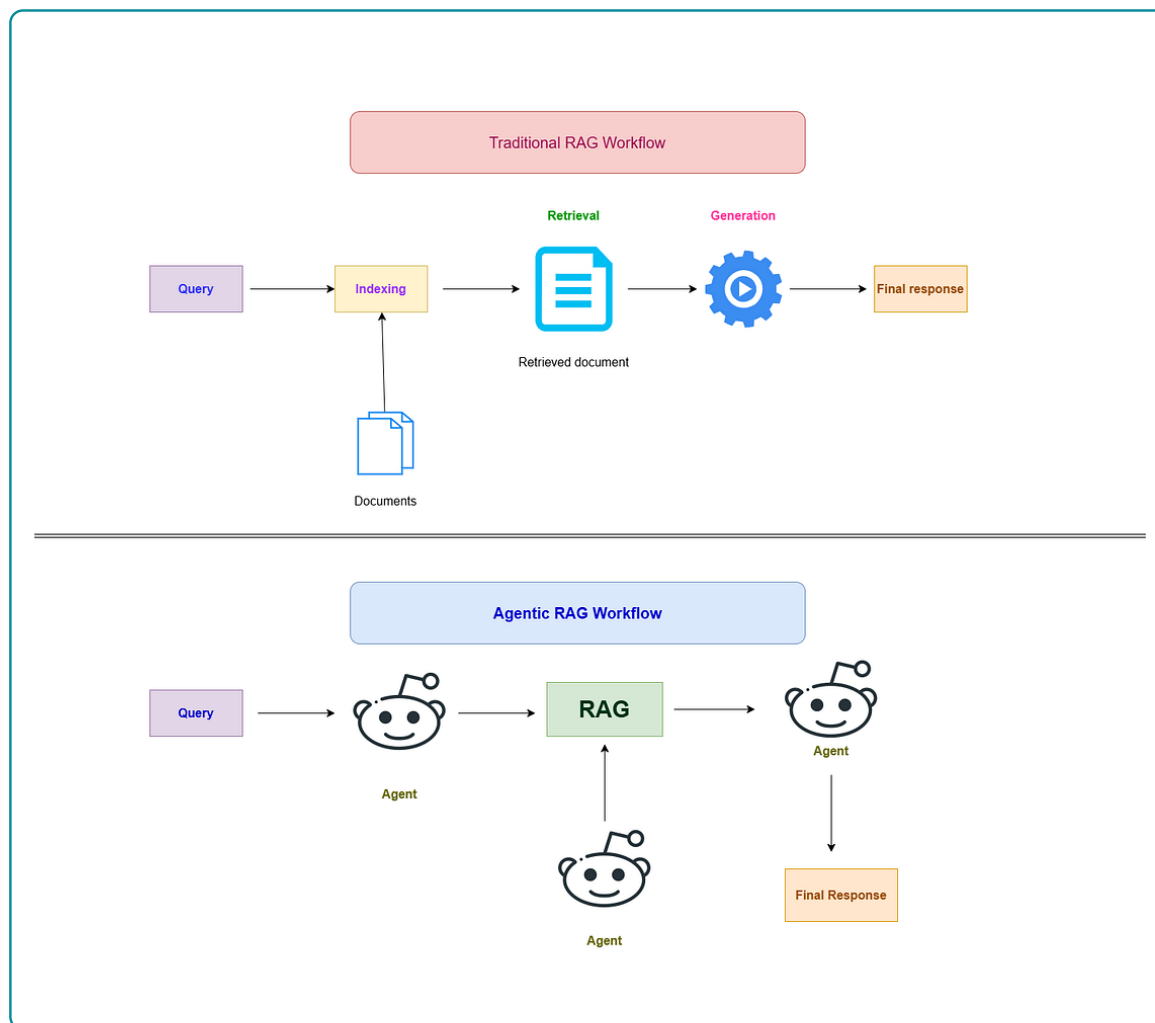
le fond.

Dans un système de recommandation emploi-compétences, une RAG purement vectorielle permettrait par exemple de récupérer les offres les plus proches du profil d'un candidat, puis de générer une explication en langage naturel. Cette solution améliore la lisibilité de la recommandation, mais elle reste insuffisante pour juger la compatibilité professionnelle. Deux profils peuvent être proches sémantiquement tout en étant séparés par un niveau de qualification, une contrainte de localisation, une compétence obligatoire ou un domaine de formation. Une offre peut aussi ressembler lexicalement à un profil sans être réellement accessible au candidat.

Le GraphRAG répond à cette limite en introduisant un graphe de connaissances dans la boucle de récupération. Au lieu de transmettre seulement des textes proches, le système récupère également des relations explicites : un candidat possède certaines compétences, une offre en exige d'autres, un métier est associé à des compétences, une formation prépare à un métier, un niveau NCF contraint l'accès à certaines offres, et un groupe MEPC situe le métier dans une nomenclature nationale. Le mécanisme change donc la nature du contexte : la récupération ne porte plus seulement sur des documents similaires, mais aussi sur des chemins de graphe, des voisins, des contraintes et des relations typées. Le graphe permet ainsi de passer d'une proximité textuelle à une explication relationnelle. Il ne se contente pas de dire que deux textes sont proches ; il peut indiquer par quels chemins ils le sont, et quelles contraintes restent bloquantes.

Concrètement, une requête du type « recommander une offre à ce candidat » peut suivre deux voies complémentaires. La voie vectorielle retrouve les offres proches du profil dans l'espace des embeddings. La voie graphe vérifie ensuite les relations explicites : niveau de formation requis, compétences possédées, compétences manquantes, métier visé, secteur et formations possibles. Le GraphRAG assemble ces deux familles d'information avant la génération. La réponse finale peut donc expliquer qu'une offre est retenue non seulement parce qu'elle est proche textuellement, mais aussi parce que le candidat possède plusieurs compétences clés, que le niveau NCF est compatible et que les compétences manquantes peuvent être couvertes par des formations identifiées.

Pour le présent projet, cette distinction est déterminante. Le modèle d'embeddings fine-tuné sert à produire une première liste d'offres candidates par similarité sémantique. Neo4j sert ensuite à contrôler cette liste par les relations métier-compétence-formation. Enfin, la couche générative cible doit verbaliser les résultats sans inventer les justifications. Le LLM intervient donc comme un rédacteur contrôlé par le contexte, et non comme une autorité autonome sur la compatibilité professionnelle.

FIGURE 1.4 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG

Source : Figure reprise du dossier du projet, inspirée de l'article Medium sur l'Agentic RAG [33]

La figure 1.4 illustre la rupture entre une RAG traditionnelle et une architecture agentic. Dans le flux traditionnel, la requête traverse une chaîne relativement fixe : indexation, récupération d'un document, génération, puis réponse finale. Dans le flux agentic, la requête passe par un ou plusieurs agents capables de coordonner la récupération, de solliciter plusieurs outils et d'organiser la réponse. L'image met surtout en évidence que l'Agentic RAG n'est pas seulement une meilleure recherche documentaire ; c'est une organisation dynamique de la recherche et de la génération.

1.7 Agentic RAG

L'Agentic RAG désigne une extension de la RAG dans laquelle la récupération n'est plus une étape fixe et unique, mais une séquence pilotée par un agent. L'idée, mise en avant dans les travaux récents sur les agents outillés et illustrée dans des synthèses techniques comme l'article Medium consulté [33], est de remplacer une chaîne linéaire « requête–recherche–génération » par une boucle plus adaptative : planifier, appeler un outil, observer le résultat, décider si l'information est suffisante, puis continuer ou produire la réponse. Cette logique rejoint les modèles de raisonnement-action tels que ReAct, où un modèle alterne raisonnement et usage d'outils externes [34].

La différence avec une RAG classique est importante. Dans une RAG standard, la stratégie de récupération est déterminée à l'avance : le système récupère, par exemple, les dix documents les plus proches puis génère une réponse. Dans un Agentic RAG, le système peut décider que les documents récupérés sont insuffisants, reformuler la requête, interroger une autre base, vérifier une contrainte ou demander un calcul complémentaire. Des approches comme Self-RAG montrent également l'intérêt d'introduire des mécanismes de critique ou d'auto-vérification afin de décider quand récupérer de l'information et comment contrôler la génération [35].

Son fonctionnement repose donc sur quatre composantes. La première est un planificateur, qui transforme l'objectif utilisateur en étapes : chercher, vérifier, comparer, expliquer. La deuxième est une bibliothèque d'outils, par exemple une base vectorielle, un graphe de connaissances, un module de scoring ou une API métier. La troisième est une mémoire ou une trace d'exécution, qui conserve les observations produites à chaque étape. La quatrième est un module de contrôle, qui vérifie si la réponse est suffisamment fondée avant de la présenter. Sans ce contrôle, l'agentivité devient un risque : le système peut multiplier les actions sans améliorer la vérité de la réponse.

Dans le cadre du matching emploi-compétences, l'Agentic RAG ne doit pas être compris comme un agent libre qui « pense » à la place de l'expert. Ce serait méthodologiquement fragile. Il doit plutôt être conçu comme une orchestration contrôlée de fonctions spécialisées. Le système peut, par exemple, recevoir l'objectif suivant : recommander des offres accessibles à un candidat et proposer une trajectoire de montée en compétences. L'agent transforme alors cet objectif en un plan opérationnel :

1. récupérer le profil normalisé du candidat ;
2. rechercher les offres proches dans la base vectorielle `pgvector` ;
3. vérifier dans Neo4j les contraintes de niveau NCF, de secteur, de localisation et de compétences ;
4. calculer le *skill gap* pour les offres présélectionnées ;
5. pénaliser les offres présentant trop de compétences essentielles manquantes ;
6. générer une justification fondée sur les scores, les relations du graphe et les formations associées.

La plus-value est donc différente de celle d'un simple modèle de langue. Un Agentic RAG apporte une capacité d'orchestration : il combine recherche sémantique, interrogation du graphe, calcul de score, contrôle des contraintes et verbalisation finale. Dans un système emploi-formation, cette orchestration est précieuse parce que la bonne recommandation n'est pas nécessairement l'offre textuellement la plus proche. Elle est l'offre dont la proximité sémantique, la compatibilité institutionnelle, les compétences requises et les possibilités de formation forment un ensemble cohérent.

Pour éviter l'effet de mode, il faut toutefois poser des limites claires. Un Agentic RAG n'améliore pas automatiquement la qualité d'un système. S'il appelle de mauvais outils, s'il dispose d'un graphe incomplet ou s'il n'enregistre pas ses décisions, il peut produire des recommandations plus complexes mais pas plus fiables. Dans ce mémoire, l'agentivité doit donc être envisagée comme une couche cible d'orchestration explicable : les scores restent calculés par le code, les contraintes sont vérifiées dans les données, les chemins de graphe sont traçables, et le LLM sert à organiser et formuler la réponse. Cette position est plus défendable qu'une présentation vague où l'agent serait supposé prendre de meilleures décisions par lui-même.

Appliquée au projet, la logique Agentic RAG peut être résumée ainsi : le modèle d'embeddings propose

des candidats, le graphe vérifie et explique, le module de scoring classe, le générateur rédige, et l'agent coordonne la séquence en fonction de l'objectif. La valeur ajoutée scientifique tient alors à l'articulation entre représentation vectorielle, raisonnement relationnel et contrôle génératif; la valeur ajoutée opérationnelle tient à la capacité d'expliquer non seulement quelle offre est recommandée, mais aussi pourquoi elle est accessible, quelles compétences manquent et quelles formations peuvent réduire cet écart.

APPROCHE MÉTHODOLOGIQUE ET DONNÉES

2.1 Positionnement méthodologique

La présente étude adopte une démarche d'ingénierie appliquée orientée vers la conception, l'intégration et l'évaluation d'un artefact numérique d'aide à la décision. L'objet étudié n'est donc pas uniquement un modèle statistique pris isolément, mais un système complet de recommandation emploi-compétences combinant données structurées, représentations vectorielles, graphe de connaissances, règles métier et génération augmentée par récupération. Ce positionnement est cohérent avec la problématique du mémoire : améliorer l'appariement entre candidats, offres d'emploi, compétences, métiers et trajectoires de formation dans un contexte où l'information est hétérogène, incomplète et dispersée.

La logique méthodologique retenue repose sur quatre principes. Le premier est la *traçabilité* : chaque recommandation doit pouvoir être rattachée à des données observables, à un référentiel ou à une relation explicite du graphe. Le deuxième est l'*hybridation* : aucune source d'information ne suffit seule à résoudre le problème. La similarité sémantique capte les proximités de langage, le graphe encode les relations métier, les règles contrôlent les contraintes de niveau et le LLM sert à formuler une réponse intelligible. Le troisième principe est l'*explicabilité opérationnelle* : le système ne doit pas seulement retourner une liste d'offres, mais justifier le rapprochement, signaler les compétences communes, identifier les écarts et proposer une trajectoire de montée en compétences. Le quatrième principe est le *contrôle génératif* : les réponses produites par le LLM doivent être contraintes par le contexte récupéré afin de réduire le risque d'hallucination, conformément à la logique du RAG [21, 36].

Ce chapitre précise donc les choix de conception du système. Les résultats d'exécution, les performances mesurées et les exemples de réponses sont volontairement réservés aux chapitres suivants. La méthodologie décrit ici comment les données sont préparées, comment les représentations sont construites, comment les bases pgvector et Neo4j sont articulées, comment le workflow agentique est organisé et comment l'évaluation est conçue.

TABLE 2.1 – Correspondance entre objectifs spécifiques et choix méthodologiques

Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d'une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d'embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d'un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top- K [13, 18].
Représenter les relations entre métiers, compétences et profils	Construction d'un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [19, 20].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l'évaluation du retrieval, du graphe, du classement et de la réponse générée.

Source : Auteur

2.2 Sources de données et leur rôle dans le système

Le système mobilise deux catégories de données. La première correspond aux données opérationnelles du marché du travail : offres d'emploi et profils de candidats. Ces données décrivent respectivement la demande de travail et l'offre de travail disponible. La seconde catégorie correspond aux référentiels de structuration : ESCO, MEPC et NCF. Ces référentiels permettent d'éviter une approche purement textuelle et de rattacher les intitulés, compétences et niveaux de formation à des nomenclatures interprétables.

Les offres d'emploi apportent les informations relatives aux postes disponibles : intitulé, employeur, secteur, localisation, type de contrat, expérience attendue, niveau d'étude, compétences mentionnées et description détaillée. Les profils candidats décrivent les caractéristiques individuelles : diplôme, niveau d'étude, spécialité, expérience, compétences, mobilité et objectif professionnel. Ces deux sources sont naturellement bruitées : les intitulés ne sont pas normalisés, les compétences peuvent être formulées librement et certaines variables sont incomplètes. La méthodologie doit donc transformer ces données en objets comparables.

ESCO fournit une nomenclature internationale des métiers et compétences. Son intérêt est de disposer d'un vocabulaire structuré et de relations métier-compétence déjà établies. La MEPC permet d'ancrer le système dans la nomenclature camerounaise des métiers. La NCF fournit une structure nationale des niveaux et domaines de formation. L'intégration conjointe de ces sources permet de relier le langage des annonces, les profils des candidats et les classifications institutionnelles.

TABLE 2.2 – Sources de données mobilisées et utilisation méthodologique

Source	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé; création des embeddings candidats; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature camerounaise des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature camerounaise des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

Source : Auteur

2.3 Préparation et normalisation des données

La préparation des données constitue la première couche méthodologique du système. Son objectif est de produire des tables propres, stables et exploitables par les deux moteurs de recherche : le moteur vectoriel et le moteur graphe. Cette étape ne se limite pas au nettoyage cosmétique des textes. Elle détermine la qualité des relations construites, la pertinence des embeddings et la fiabilité des recommandations.

Pour les offres d'emploi, la normalisation consiste d'abord à réduire les doublons et les variations de forme. Une annonce peut apparaître plusieurs fois avec des différences mineures de titre, d'employeur ou de localisation. Le traitement doit donc stabiliser les identifiants et conserver les informations utiles sans multiplier artificiellement les observations. Les champs textuels sont ensuite harmonisés : suppression des caractères parasites, standardisation des espaces, séparation des champs multi-valeurs et conservation des informations de contexte. Les compétences mentionnées sont transformées en listes exploitables, tandis que les localisations et secteurs sont séparés en valeurs principales et valeurs secondaires.

Pour les candidats, la normalisation porte sur le diplôme, le niveau d'étude, la spécialité, l'expérience, les compétences, le secteur visé et la mobilité géographique. Une attention particulière est portée au niveau de formation. Lorsque l'information disponible le permet, le niveau est rapproché de la NCF. Ce rapprochement est indispensable parce qu'une recommandation emploi-compétences ne peut pas se fonder uniquement sur la proximité textuelle : une offre peut être sémantiquement proche d'un profil tout en exigeant un niveau de qualification supérieur.

Chaque offre et chaque candidat reçoit enfin un champ `text_to_embed`. Ce champ synthétise les éléments utiles pour la recherche sémantique : intitulé, compétences, secteur, niveau, description et ob-

jectif. Le choix d'un texte synthétique répond à une contrainte pratique : les modèles d'embeddings travaillent sur des séquences textuelles, mais les données métier sont partiellement structurées. Le champ `text_to_embed` sert donc d'interface entre les variables métiers et l'espace vectoriel.

2.4 Alignement des référentiels métiers et formations

L'alignement des référentiels répond à un besoin central : rendre comparables des données issues de sources différentes. Les annonces utilisent des formulations libres, ESCO propose une taxonomie internationale, la MEPC suit une logique nationale de classification des métiers et la NCF structure les formations. La méthode adoptée est prudente : elle vise un alignement exploitable pour la recommandation, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

La MEPC est structurée selon ses niveaux hiérarchiques : grands groupes, sous-groupes et groupes de base. La NCF est organisée autour des niveaux de formation, grands domaines, domaines spécialisés et domaines détaillés. Cette structuration permet de conserver l'information institutionnelle au lieu de la réduire à de simples libellés. Les tables obtenues alimentent ensuite le graphe Neo4j et la base vectorielle.

L'alignement avec ESCO s'appuie sur les codes et les proximités de libellés lorsque ces informations sont disponibles. Cette stratégie est méthodologiquement raisonnable, mais elle doit être interprétée avec prudence. Un même groupe de classification peut regrouper plusieurs réalités professionnelles. L'alignement sert donc à améliorer la couverture et l'interprétabilité, non à produire une vérité définitive sur l'équivalence entre métiers.

2.5 Choix du modèle d'embeddings et fine-tuning

Le système utilise un modèle SentenceTransformer pour représenter les offres, candidats, métiers, compétences et fragments documentaires dans un espace vectoriel. Ce choix est adapté au problème, car le matching emploi-compétences repose largement sur la similarité sémantique entre textes courts ou semi-structurés : intitulés de postes, compétences, descriptions d'annonces, objectifs professionnels et libellés de référentiels. Les SentenceTransformers ont été conçus pour produire des embeddings directement comparables par similarité cosinus, ce qui les rend adaptés aux tâches de recherche sémantique [13].

Le fine-tuning retenu n'est pas un fine-tuning génératif. Il ne vise pas à apprendre au modèle à rédiger des réponses, mais à adapter l'espace vectoriel au vocabulaire local du domaine emploi-compétences. La logique est contrastive : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Par exemple, une offre de « data analyst » doit être rapprochée d'un profil mentionnant analyse de données, statistique, Python, SQL ou visualisation, même si les formulations exactes diffèrent. A l'inverse, un profil orienté développement web ne doit pas être considéré comme équivalent à une offre de comptabilité uniquement parce que certaines expressions génériques se ressemblent.

Les paires d'apprentissage sont construites à partir des données préparées. Elles peuvent mobiliser les relations offre-compétence, métier-compétence, candidat-métier et offre-candidat lorsque les signaux disponibles sont suffisamment fiables. Cette étape est critique : un fine-tuning apprend les régularités de ses exemples. Des paires mal construites peuvent dégrader l'espace vectoriel au lieu de l'améliorer. La méthodologie retient donc une construction contrôlée des exemples, puis une évaluation séparée du modèle de base et du modèle adapté.

2.6 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings produits sont stockés dans PostgreSQL avec l’extension pgvector. Ce choix permet de conserver dans un même environnement des représentations vectorielles et des métadonnées relationnelles. pgvector permet l’indexation et la recherche de voisins proches dans PostgreSQL, ce qui facilite l’intégration avec les autres tables du système [18]. Pour accélérer les recherches top- K , la méthode s’appuie sur une logique d’indexation approximative des voisins proches, cohérente avec les travaux sur HNSW [16].

La base vectorielle joue deux rôles. Le premier est le retrieval sémantique : retrouver les offres, métiers, compétences ou documents proches d’une requête. Le second est le préfiltrage pour la recommandation : produire un ensemble de candidats plausibles avant d’appliquer des contraintes plus structurées. Ce préfiltrage est nécessaire, car le graphe de connaissances apporte de l’explicabilité, mais il n’est pas toujours le meilleur premier filtre lorsque la requête est formulée librement.

Chaque résultat vectoriel doit conserver ses métadonnées : identifiant, type d’entité, libellé, source, score de similarité et texte associé. Sans ces métadonnées, la génération finale serait incapable de citer ses sources et le système perdrait sa traçabilité.

2.7 Construction du graphe de connaissances Neo4j

Le graphe de connaissances représente la couche relationnelle du système. Son rôle est de modéliser explicitement les relations entre les candidats, les offres, les compétences, les métiers, les secteurs, les localisations et les référentiels. Cette représentation est justifiée par la nature même du problème : le matching emploi-compétences n’est pas seulement une question de proximité textuelle, mais une question de relations. Une offre requiert des compétences, un candidat possède des compétences, un métier est classé dans une nomenclature, un niveau de formation conditionne l’accès à certains postes, et une trajectoire de montée en compétences dépend des écarts identifiés.

La méthodologie retient des noeuds correspondant aux entités principales : `OffreEmploi`, `Candidat`, `Compétence`, `Métier`, `Secteur`, `Localisation`, `DocChunk`, `DocumentReferentiel` et groupes de nomenclature. Les relations décrivent les liens fonctionnels : une offre `REQUIERT` une compétence, un candidat `POSSEDE` une compétence, une offre est rattachée à un secteur ou à une localisation, un métier est classé dans un groupe, et un fragment documentaire provient d’un document référentiel.

Ce graphe sert à trois opérations méthodologiques. Premièrement, il vérifie les correspondances proposées par la recherche vectorielle. Deuxièmement, il explique les recommandations en identifiant les compétences communes et manquantes. Troisièmement, il permet des requêtes relationnelles que la recherche vectorielle gère mal, par exemple : quelles compétences sont associées à un métier donné ? quelles offres exigent une compétence précise ? quelles compétences manquent à un candidat pour viser une famille de métiers ?

2.8 Score hybride et logique de skill gap

Le score de recommandation combine plusieurs signaux complémentaires. La similarité sémantique mesure la proximité entre le texte du profil et celui de l’offre. La couverture de compétences mesure la proportion des compétences requises par l’offre qui sont déjà présentes chez le candidat. La compatibilité de niveau vérifie si le niveau de formation du candidat est cohérent avec le niveau attendu par l’offre.

L'alignement sectoriel mesure la cohérence entre le secteur ou le domaine du candidat et celui de l'offre.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture de compétences issue du graphe, C_{ncf} la compatibilité de niveau de formation et A_{sect} l'alignement sectoriel. Cette formulation traduit un choix méthodologique important : le système ne délègue pas toute la décision au LLM. Le LLM intervient pour synthétiser et expliquer, mais le classement repose sur des signaux calculables et auditables.

L'analyse de *skill gap* complète le score. Elle distingue les compétences acquises, les compétences requises et les compétences manquantes. Cette distinction permet de produire un verdict opérationnel : candidat immédiatement compatible, candidat compatible après montée en compétence, candidat à conserver en vivier ou profil hors cible. Ce verdict est plus utile qu'un simple score numérique, car il transforme la recommandation en aide à la décision.

2.9 Architecture Agentic RAG et orchestration LangGraph

L'orchestration du système repose sur un workflow agentique. La requête utilisateur est d'abord analysée afin d'identifier son intention : diagnostic, recommandation candidat, skill gap, recherche référentielle, question relationnelle sur le graphe, orientation métier ou recherche générale. Le système sélectionne ensuite les outils adaptés. Cette étape de routage est essentielle : interroger pgvector, Neo4j ou les documents réglementaires ne répond pas au même besoin.

Le workflow suit une chaîne contrôlée : `analyse_request`, `plan_tools`, `execute_tools`, `build_context`, `generate_final_answer`, puis critique éventuelle de la réponse. Le noeud d'analyse identifie le use-case. Le noeud de planification sélectionne les outils. Le noeud d'exécution appelle les bases et services nécessaires. Le noeud de construction de contexte organise les résultats récupérés. Le noeud de génération produit la réponse finale avec le LLM. Le critic vérifie ensuite si la réponse paraît suffisamment fidèle au contexte ; en cas de faiblesse, le workflow peut élargir le contexte avant de régénérer la réponse.

Ce choix s'inscrit dans la logique de l'Agentic RAG : le système n'est pas un simple prompt envoyé à un modèle, mais une orchestration d'outils spécialisés autour d'un état partagé [34, 35]. Dans le présent travail, l'agent n'est pas conçu comme une entité autonome non contrôlée. Il est plutôt un routeur raisonné, limité à des outils définis, dont les sorties sont tracées.

2.10 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Le premier concerne l'interrogation du graphe en langage naturel. Pour les questions de type `graph_query` ou les recherches générales nécessitant un raisonnement relationnel, la question peut être transformée en requête Cypher par un module Text2Cypher. Le modèle retenu pour cette tâche est `neo4j/text2cypher-gemma-2-9b-it-finetuned-2024v1`. La génération de Cypher est encadrée par le schéma Neo4j : types de noeuds, propriétés et relations autorisées. Cette contrainte est indispensable, car un modèle génératif qui ne connaît pas le schéma réel peut produire des requêtes syntaxiquement plausibles mais inexécutables ou non pertinentes.

Le second usage du LLM concerne la génération de la réponse finale. Le système utilise l'API OpenRouter avec le modèle `openai/gpt-oss-20b:free`. Ce modèle ne doit pas inventer la réponse à partir de ses connaissances générales. Il reçoit un contexte construit par le workflow : résultats pgvector, lignes Neo4j, fragments documentaires et métadonnées de source. Le prompt lui demande de répondre en français, de structurer la réponse et de citer les informations utilisées.

Cette séparation des rôles est méthodologiquement importante. Text2Cypher sert à produire une requête structurée vers le graphe. Le LLM final sert à formuler une synthèse lisible. Dans les deux cas, le modèle est subordonné aux données disponibles et non l'inverse.

2.11 Interfaces et traçabilité

Le système prévoit trois modes d'accès : une interface Streamlit pour l'usage conversationnel, une API FastAPI pour l'intégration applicative et une CLI pour les tests de développement. Ces interfaces ne changent pas la logique de recommandation ; elles exposent le même moteur sous des formes différentes. La CLI facilite la vérification rapide des use-cases, Streamlit permet une démonstration interactive et FastAPI prépare l'intégration dans un environnement applicatif.

La traçabilité est intégrée à plusieurs niveaux. Les traces du workflow indiquent les noeuds traversés et les outils appelés. Les résultats conservent les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. La réponse finale doit citer les éléments utilisés. Cette exigence est décisive dans un système d'aide à l'orientation ou au recrutement : une recommandation sans preuve peut être séduisante, mais elle n'est pas défendable.

2.12 Protocole d'évaluation prévu

L'évaluation est conçue comme une évaluation en plusieurs étages. Le premier étage porte sur le retrieval vectoriel : le système retrouve-t-il les offres, compétences, métiers ou documents attendus dans les premiers résultats ? Les métriques retenues sont notamment Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K, couramment utilisées pour les tâches de recherche d'information [37, 38].

Le deuxième étage porte sur le graphe : les relations retournées sont-elles correctes, utiles et interprétables ? Les tests doivent couvrir les requêtes métier-compétence, candidat-compétence, offre-compétence, compatibilité de niveau et recherche de chemins explicatifs. Le troisième étage porte sur le score hybride : il s'agit de comparer les classements produits par la recherche vectorielle seule, le graphe seul et la combinaison hybride. Le quatrième étage porte sur la réponse générée : fidélité au contexte, pertinence par rapport à la question, couverture des preuves, citations et utilité opérationnelle.

La méthode distingue donc clairement performance de récupération et qualité de génération. Une réponse bien rédigée ne prouve pas que le système a retrouvé les bonnes informations. Inversement, un bon retrieval ne garantit pas une bonne synthèse. Cette séparation protège l'analyse contre une erreur fréquente dans les systèmes RAG : juger le système uniquement à l'apparence de la réponse finale.

2.13 Limites méthodologiques et précautions d'interprétation

Plusieurs limites doivent être prises en compte dès la conception. Premièrement, les données d'offres et de candidats peuvent contenir des omissions, des doublons et des formulations ambiguës. Deuxièmement, l'alignement entre référentiels reste partiel : il améliore la structuration, mais ne garantit pas une

équivalence parfaite entre catégories. Troisièmement, le graphe dépend de la qualité des relations extraites ou importées. Une relation manquante peut conduire à sous-estimer une compatibilité réelle. Quatrièmement, les LLM peuvent produire des formulations plausibles mais non soutenues par les données ; c'est pourquoi le contexte, les citations et le critic sont nécessaires.

Une autre limite est matérielle. L'utilisation locale ou semi-locale de modèles spécialisés, notamment pour Text2Cypher, peut être contrainte par la mémoire disponible. Cette contrainte influence les choix techniques : recours à une API pour la génération finale, chargement contrôlé du modèle Text2Cypher et possibilité de fallback vers des templates Cypher sécurisés. Ces choix ne sont pas des faiblesses méthodologiques en soi ; ils traduisent l'adaptation du système aux ressources disponibles.

2.14 Synthèse de l'approche méthodologique

La méthodologie proposée construit un système hybride plutôt qu'un modèle isolé. Elle part de données locales hétérogènes, les normalise, les rattache à des référentiels, les encode dans un espace vectoriel, les structure dans un graphe de connaissances, puis orchestre leur interrogation par un workflow agentique. Le rôle du LLM est strictement encadré : transformer certaines questions en requêtes graphe, puis formuler une réponse fondée sur les preuves récupérées.

Cette approche répond aux objectifs du mémoire. Elle permet de dépasser la recherche par mots-clés, d'exploiter les référentiels institutionnels, de produire des recommandations explicables et de relier chaque verdict à des compétences, des niveaux de formation et des sources identifiables. Les chapitres suivants présentent la réalisation opérationnelle de cette méthodologie, puis l'évaluation de ses performances.

Deuxième partie

Présentation des résultats

IMPLÉMENTATION DU SYSTÈME DE RECOMMANDATION

3.1 Vue d'ensemble de l'implémentation

Ce chapitre présente la réalisation opérationnelle du système de recommandation hybride emploi-compétences. Il ne reprend pas les fondements théoriques du chapitre précédent ; il décrit plutôt comment les choix méthodologiques ont été traduits en modules exécutables, bases de données, outils d'interrogation et interfaces utilisateur. L'implémentation suit une logique modulaire : chaque brique remplit une fonction précise, mais l'ensemble est orchestré dans un workflow agentique capable de router les requêtes vers les sources pertinentes.

Le système est organisé autour de huit blocs fonctionnels. Le premier bloc correspond à l'ETL et à la normalisation des données. Le deuxième concerne le fine-tuning du modèle SentenceTransformer. Le troisième construit le graphe de connaissances dans Neo4j. Le quatrième matérialise les embeddings dans PostgreSQL avec l'extension pgvector. Le cinquième implémente le moteur GraphRAG et le score hybride. Le sixième ajoute le pipeline Text2Cypher pour interroger le graphe en langage naturel. Le septième orchestre l'ensemble avec LangGraph. Le huitième expose le système à travers une CLI, une interface Streamlit et une API FastAPI.

TABLE 3.1 – Organisation opérationnelle des modules implémentés

Bloc	Répertoire ou fichier principal	Rôle dans le système
ETL et normalisation	<code>src/01_etl</code>	Nettoyage des offres, candidats, référentiels et construction des champs d'embedding.
Fine-tuning ST	<code>src/02_finetune_st</code>	Adaptation du modèle SentenceTransformer au vocabulaire emploi-compétences.
Graphe Neo4j	<code>src/03_knowledge_graph</code>	Création des noeuds, relations, contraintes et requêtes Cypher métier.
pgvector	<code>src/04_pgvector</code>	Stockage des embeddings, recherche ANN et recherche hybride dense-lexicale.
GraphRAG	<code>src/05_graphrag</code>	Construction du contexte candidat-offre, score hybride, critic et Text2Cypher.
API	<code>src/06_api</code>	Exposition des endpoints de recommandation, skill gap, embedding et chat.
Evaluation	<code>src/07_evaluation</code>	Scripts de calcul des métriques de retrieval, embeddings et fidélité.
Agentic RAG	<code>src/08_agentic_graphrag</code>	Orchestration LangGraph, outils, routage, génération finale et traces.
Interface UI	<code>chatbot_app.py</code>	Interface Streamlit conversationnelle avec affichage des traces du workflow.

Source : Auteur, à partir de l'architecture du projet

Cette structuration répond à une exigence de maintenabilité. Les données, les embeddings, le graphe, la génération et les interfaces ne sont pas fusionnés dans un script unique. Chaque couche peut être testée séparément, puis mobilisée par l'agent selon le besoin de l'utilisateur.

3.2 Implémentation de la chaîne ETL

La chaîne ETL est le point d'entrée du système. Elle transforme des données hétérogènes en tables normalisées exploitables par les modèles et les bases de données. Le module `src/01_etl` contient les traitements relatifs aux offres, aux candidats, aux référentiels et aux paires d'entraînement. Cette couche produit les champs structurants nécessaires à la suite du pipeline : identifiants stables, listes de compétences, secteurs normalisés, localisations, niveaux de formation, textes d'embedding et métadonnées.

Pour les offres d'emploi, l'implémentation est portée par `normalize_offres.py`. Le traitement commence par la lecture des données brutes, puis applique une déduplication, un nettoyage des champs textuels, une normalisation des secteurs, des localisations et des compétences. Le fichier construit ensuite deux représentations complémentaires : `text_to_embed`, utilisé pour l'encodage sémantique, et `metadata_str`, utilisé pour conserver les informations courtes de contexte. Cette séparation est importante : le texte d'embedding doit être riche, tandis que les métadonnées doivent rester facilement lisibles et filtrables.

Pour les candidats, `normalize_candidats.py` suit une logique proche. Le script harmonise les niveaux d'étude, les diplômes, les spécialités, les secteurs visés, la mobilité et les objectifs professionnels. Il produit notamment un niveau de formation rapproché de la NCF lorsque l'information disponible le permet. Le champ `text_to_embed` candidat devient la représentation textuelle utilisée pour comparer le profil à des offres, métiers ou compétences.

L'alignement des référentiels est géré par `align_referentiels.py`. Le script prépare les tables MEPC et NCF, construit les champs d'embedding associés et produit les correspondances nécessaires entre référentiels et graphe. L'enjeu opérationnel est double : préserver la hiérarchie des nomenclatures et rendre leurs libellés exploitables par la recherche vectorielle.

3.3 Implémentation du fine-tuning SentenceTransformer

Le modèle d'embeddings du projet est implémenté dans `src/02_finetune_st`. Le script principal `train_sentence_transformer.py` charge les paires d'entraînement, construit les jeux de validation et de test, puis entraîne un modèle SentenceTransformer adapté au domaine. La logique retenue est celle du fine-tuning contrastif : le modèle apprend à rapprocher les textes qui doivent être considérés comme compatibles et à éloigner ceux qui ne le sont pas.

Le résultat du fine-tuning est stocké dans `models/st_finetuned/final`. Ce chemin est ensuite utilisé par les autres composants du système, notamment les modules pgvector, GraphRAG, API et agent LangGraph. Lorsque ce modèle est disponible, il devient l'encodeur principal. En cas d'indisponibilité, certains modules prévoient un repli vers un modèle SentenceTransformer de base, mais cette situation correspond à un mode dégradé.

L'évaluation du modèle d'embeddings est séparée de son entraînement. Le fichier `evaluate_st.py` permet de comparer le modèle fine-tuné à un modèle de base. Cette séparation évite de confondre l'entraînement et la preuve de performance. Les résultats chiffrés sont présentés dans le chapitre d'évaluation ; le présent chapitre retient seulement que l'encodeur fine-tuné est intégré comme composant central du système.

3.4 Implémentation de la base vectorielle pgvector

La base vectorielle est implémentée dans le module `src/04_pgvector`. Elle utilise PostgreSQL et l'extension pgvector pour stocker les représentations vectorielles des offres, candidats, compétences, métiers, groupes MEPC et domaines NCF. Le script `embed_all_entities.py` charge le modèle SentenceTransformer, encode les textes préparés par l'ETL et insère les vecteurs dans la table d'embeddings.

Chaque ligne indexée conserve plusieurs informations : le type d'entité, l'identifiant métier, la source, le libellé, le texte encodé, le vecteur, l'identifiant du modèle et, lorsque c'est possible, l'identifiant du noeud Neo4j correspondant. Cette dernière colonne assure la passerelle entre pgvector et le graphe. Elle permet de partir d'un résultat de recherche sémantique et de retrouver ensuite le noeud relationnel associé.

La recherche vectorielle est complétée par une recherche lexicale et une fusion des scores. Le module `hybrid_search.py` implémente une recherche dense par embedding, une recherche textuelle et une fusion de rangs. Cette hybridation réduit le risque de manquer une entité lorsque la requête contient un terme exact important, mais dont l'embedding seul ne suffit pas à le faire remonter.

L'instance courante vérifiée par le diagnostic du système contient les volumes suivants dans pgvector.

TABLE 3.2 – Entités indexées dans pgvector au moment du diagnostic

Type d'entité	Nombre	Rôle dans le retrieval
OFFRE_EMPLOI	15 722	Corpus principal de recommandation d'offres.
COMPETENCE	13 939	Recherche et enrichissement des compétences.
METIER	3 039	Orientation métier et rapprochement avec les profils.
CANDIDAT	1 105	Recherche candidat-offre et comparaison de profils.
GRUPE_BASE_MEPC	209	Ancrage métier national.
DOMAINE_DETAILLE_NCF	201	Ancrage formation national.

Source : Diagnostic local du système

3.5 Implémentation du graphe de connaissances Neo4j

La construction du graphe est assurée par le module `src/03_knowledge_graph`, principalement par le script `load_neo4j.py`. Ce script charge les référentiels, les offres, les candidats et les relations métier-compétence dans Neo4j. Il peut être exécuté en pipeline complet ou par étape : schéma, ESCO, MEPC, NCF, offres, candidats, relations et compétences. Cette granularité facilite les reprises partielles et limite le coût des rechargements.

Le graphe contient les entités opérationnelles du système : `OffreEmploi`, `Candidat`, `Compétence`, `Métier`, `Secteur`, `Localisation`, ainsi que les noeuds de référentiels MEPC et NCF. Les relations structurantes permettent de représenter explicitement les exigences d'une offre, les compétences possédées par un candidat, les rattachements sectoriels, les localisations et les classifications métiers.

Le diagnostic exécuté sur l'instance locale indique que Neo4j contient 43 008 noeuds. Ce chiffre ne constitue pas une métrique de qualité, mais il atteste que la base graphe utilisée par le système est effectivement alimentée. La qualité du graphe dépend ensuite de la cohérence des relations et de leur capacité à soutenir les scénarios de recommandation, ce qui relève du chapitre d'évaluation.

TABLE 3.3 – Principales fonctions du graphe dans le système

Fonction	Utilisation opérationnelle
Vérification relationnelle	Contrôler si une compétence, un métier ou un secteur est relié au candidat ou à l'offre.
Explication du matching	Identifier les compétences communes et les compétences manquantes entre un candidat et une offre.
Recherche graphe	Répondre aux questions relationnelles : compétences d'un métier, offres liées à une compétence, profils proches.
Contrainte institutionnelle	Mobiliser les niveaux et domaines NCF ainsi que les groupes MEPC dans l'interprétation.
Traçabilité	Rattacher les réponses à des noeuds et relations explicites.

Source : Auteur

3.6 Implémentation du moteur de recommandation hybride

Le moteur de recommandation est porté par le module `src/05_graphrag`. La classe principale `RecommendationEngine` combine un constructeur de contexte, un appel au LLM et un format de sortie structuré. Elle utilise la recherche pgvector comme premier filtre, puis enrichit les offres candidates par des informations issues de Neo4j.

Le constructeur de contexte, implémenté dans `context_builder.py`, suit une chaîne en plusieurs étapes. Il récupère d'abord les offres sémantiquement proches du candidat dans pgvector. Il enrichit ensuite ces offres par le graphe : compétences requises, compétences possédées, compétences manquantes, niveau de formation, secteur et autres signaux disponibles. Il calcule enfin un score hybride et prépare un contexte textuel destiné à la génération finale.

Le score hybride implémenté combine quatre composantes : similarité sémantique, couverture graphe, compatibilité NCF et alignement sectoriel. Cette structure évite de produire une recommandation fondée uniquement sur le texte. Une offre peut être proche sémantiquement du profil, mais peu réaliste si le niveau requis est trop éloigné ou si les compétences essentielles sont absentes. Inversement, une offre peut ne pas partager exactement les mêmes mots que le profil tout en étant compatible par ses compétences et son secteur.

La logique de *skill gap* transforme le score en verdict interprétable. Le système ne se contente pas de classer les offres ; il indique les compétences déjà couvertes, les compétences manquantes et les raisons pour lesquelles une offre est immédiatement accessible ou nécessite une montée en compétence. Cette couche constitue l'une des différences majeures entre une simple recherche sémantique et un système d'aide à la décision.

3.7 Implémentation du pipeline Text2Cypher

Le module Text2Cypher est implémenté dans `src/05_graphrag/text2cypher.py`. Son rôle est de transformer certaines questions en langage naturel en requêtes Cypher exécutables sur Neo4j. Le modèle configuré pour cette tâche est `neo4j/text2cypher-gemma-2-9b-it-finetuned-2024v1`. Le pipeline re-

çoit la question de l'utilisateur, le schéma Neo4j et des consignes strictes afin de produire une requête orientée lecture.

La sécurité de l'interrogation est traitée explicitement. Les requêtes générées sont validées avant exécution afin de bloquer les instructions d'écriture ou de modification du graphe. Les opérations de type création, suppression, fusion ou mise à jour ne sont pas autorisées. Le pipeline est donc conçu comme un outil d'exploration et non comme un outil d'administration de la base.

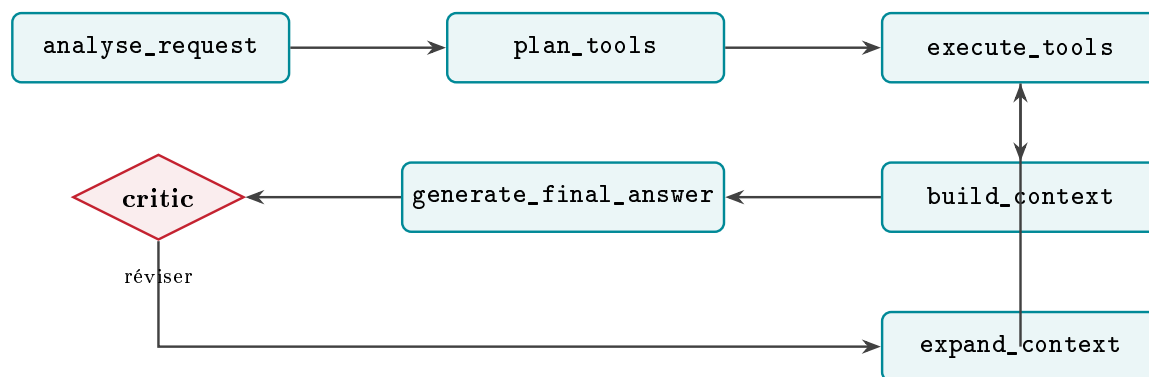
Le schéma Neo4j joue un rôle central dans ce composant. Il est transmis au modèle Text2Cypher pour l'aider à utiliser les bons labels, les bonnes propriétés et les bonnes relations. Le même schéma est également injecté dans le contexte final pour les use-cases `graph_query` et `recherche_generale`. Cette décision réduit le risque d'interprétation erronée par le LLM final, surtout lorsque la réponse doit expliquer des lignes issues du graphe.

Lorsque la génération Text2Cypher n'est pas disponible ou ne produit pas une requête satisfaisante, le système prévoit un repli vers des templates Cypher déterministes. Ces templates couvrent des intentions courantes : recherche de compétences, recherche de métier, recherche de candidat, recherche d'offre et question par défaut. Ce mécanisme ne remplace pas la flexibilité d'un modèle Text2Cypher, mais il évite qu'une panne du modèle bloque entièrement les requêtes graphe simples.

3.8 Implémentation du workflow agentique LangGraph

L'orchestration agentique est implémentée dans `src/08_agentic_graphrag/graph.py`. Le workflow LangGraph organise l'exécution autour d'un état partagé et de noeuds spécialisés. Le graphe comprend les noeuds `analyse_request`, `plan_tools`, `execute_tools`, `build_context`, `generate_final_answer` et `expand_context`. La réponse finale peut être acceptée directement ou révisée si le critic détecte une faiblesse.

FIGURE 3.1 – Workflow agentique implémenté dans LangGraph



Source : Auteur

Le noeud `analyse_request` identifie le type de demande. Le noeud `plan_tools` sélectionne les outils nécessaires. Le noeud `execute_tools` appelle ces outils et stocke les résultats. Le noeud `build_context` normalise les sorties pour la génération. Le noeud `generate_final_answer` appelle le LLM OpenRouter. Le critic vérifie ensuite la fidélité approximative de la réponse par rapport au contexte. En cas de décision `revise`, `expand_context` ajoute des outils complémentaires, puis le système relance l'exécution.

Le routage par use-case détermine les outils mobilisés. Une question strictement relationnelle active

Neo4j via Text2Cypher. Une question de recommandation candidat active le moteur hybride. Une question générale mobilise pgvector et Neo4j afin de combiner recherche sémantique et contexte relationnel. Une question de référentiel mobilise les documents indexés.

TABLE 3.4 – Outils exposés à l’agent LangGraph

Outil	Rôle
<code>service_status</code>	Vérifie les connexions pgvector, Neo4j, modèle d’embeddings et backend LLM.
<code>pgvector_semantic_search</code>	Recherche des entités métier proches d’une requête : offres, métiers, compétences.
<code>pgvector_document_search</code>	Recherche des fragments documentaires issus des référentiels indexés.
<code>neo4j_graph_query</code>	Transforme une question en Cypher read-only et interroge Neo4j.
<code>hybrid_candidate_recommendations</code>	Produit des recommandations d’offres pour un candidat via pgvector et Neo4j.
<code>global_graph_summary</code>	Calcule des indicateurs de synthèse depuis Neo4j et pgvector.

Source : Auteur, à partir du registre d’outils du système

3.9 Implémentation du LLM final et du critic

La génération finale est assurée par le client OpenRouter défini dans `src/08_agentic_graphrag/llm_client.py`. La politique du projet impose l’utilisation du modèle configuré dans l’environnement, à savoir `openai/gpt-oss-20b:free`. Le système n’utilise donc pas un modèle local via Ollama pour la réponse finale. Ce choix simplifie la contrainte matérielle locale et garantit que la génération est centralisée autour du modèle déclaré dans `OPENROUTER_MODEL`.

Le prompt final reçoit trois éléments : la question utilisateur, le contexte construit à partir des outils et les consignes de réponse. Il demande une réponse en français, structurée et limitée aux données récupérées. Pour les requêtes graphe ou générales, le contexte inclut également le schéma Neo4j afin que le modèle puisse interpréter correctement les lignes retournées par Cypher.

Le critic est implémenté dans `answer_critic.py`. Il ne s’agit pas d’un juge parfait ni d’une validation humaine. C’est un garde-fou local qui vérifie si la réponse semble suffisamment couverte par le contexte. Il calcule des signaux simples de fidélité, de couverture et de termes non supportés. Si le signal est insuffisant, LangGraph peut déclencher une révision en élargissant le contexte. Cette logique limite le risque d’une réponse fluide mais insuffisamment fondée.

3.10 API, CLI et interface Streamlit

Le système peut être utilisé de trois manières. La première est la CLI de test `src/08_agentic_graphrag/run_agent.py`. Elle permet d’envoyer une requête à l’agent et d’obtenir soit la réponse finale, soit l’état complet au format JSON. Cette CLI est utile pendant le développement, car elle expose les traces du workflow sans lancer d’interface graphique.

La deuxième manière est l’interface Streamlit `chatbot_app.py`. Elle fournit une interface conversationnelle permettant de saisir une question, d’indiquer éventuellement un identifiant candidat, de choisir le nombre de résultats et d’afficher les traces du workflow. Cette interface est adaptée à la démonstration, car elle rend visibles les noeuds et outils effectivement mobilisés par l’agent.

La troisième manière est l'API FastAPI implémentée dans `src/06_api`. Elle expose des routes pour la recommandation, le skill gap, l'encodage, le chat et la santé du système. Au démarrage, l'API initialise le modèle SentenceTransformer, la connexion Neo4j, la connexion pgvector et le moteur de recommandation. L'API prévoit également un mode dégradé lorsque certains services ne sont pas disponibles, ce qui rend le système plus robuste pendant les tests.

TABLE 3.5 – Modes d'accès au système implémenté

Interface	Commande ou point d'entrée	Usage principal
CLI	<code>run_agent.py</code>	Test rapide d'une requête et inspection de l'état final en JSON.
Streamlit	<code>streamlit run chatbot_app.py</code>	Interface conversationnelle avec visualisation des traces.
LangGraph Studio	<code>langgraph dev</code>	Visualisation du graphe agentique et exécution locale du workflow.
FastAPI	<code>src/06_api/main.py</code>	Exposition applicative des endpoints recommandation, skill gap, embed et chat.

Source : Auteur

3.11 Diagnostic d'intégration

Un diagnostic d'intégration a été exécuté sur l'instance locale afin de vérifier que les services principaux sont accessibles. La requête « diagnostic connexion pgvector neo4j » active l'outil `service_status`. Le workflow suit alors les étapes `analyse_request`, `plan_tools`, `execute_tools`, `build_context` et `generate_final_answer`. Le critic accepte la réponse, car elle est générée par un template de diagnostic.

TABLE 3.6 – Résultat du diagnostic local d'intégration

Composant	Etat observé
pgvector	Connecté; entités indexées : candidats, compétences, domaines NCF, groupes MEPC, métiers et offres.
Neo4j	Connecté; 43 008 noeuds présents dans la base.
SentenceTransformer	Modèle configuré dans <code>models/st_finetuned/final</code> .
LLM final	Backend OpenRouter actif avec <code>openai/gpt-oss-20b:free</code> .

Source : Diagnostic exécuté localement

Ce diagnostic ne mesure pas encore la qualité des recommandations. Il vérifie seulement que les briques nécessaires au fonctionnement du système sont disponibles et raccordées. La qualité du classement, la pertinence des réponses et la fidélité aux sources sont traitées dans le chapitre suivant.

3.12 Synthèse du chapitre

L'implémentation réalisée transforme l'architecture méthodologique en un système exécutable. Les offres, candidats et référentiels sont préparés par l'ETL ; les textes sont encodés par un SentenceTransformer fine-tuné ; les vecteurs sont stockés dans pgvector ; les relations métier-compétence-candidat-offre sont représentées dans Neo4j ; le moteur GraphRAG combine les deux sources ; Text2Cypher permet l'interrogation relationnelle en langage naturel ; LangGraph orchestre les outils ; OpenRouter produit la réponse finale ; Streamlit, FastAPI et la CLI exposent le système.

La force de cette implémentation tient dans son caractère hybride et traçable. Le système ne délègue pas la décision au seul LLM. Il utilise le LLM comme couche de formulation et d'interprétation, tandis que les données, les scores, les chemins du graphe et les résultats pgvector constituent la base de la recommandation. Cette séparation est essentielle pour un système destiné à l'aide au recrutement et à la montée en compétences.

ÉVALUATION DE LA PERFORMANCE DU SYSTÈME

4.1 Objectif et logique générale de l'évaluation

Ce chapitre évalue empiriquement le système implémenté au chapitre précédent. L'objectif n'est pas seulement de produire des scores globaux, mais de répondre à une question centrale du mémoire : l'ajout d'un graphe de connaissances Neo4j apporte-t-il une amélioration mesurable par rapport à une recherche vectorielle fondée uniquement sur pgvector ? Cette question est décisive, car le graphe ajoute une complexité technique réelle : modélisation des nœuds et relations, chargement Neo4j, requêtes Cypher, enrichissement des résultats, maintenance d'une base supplémentaire. Son intérêt doit donc être justifié par un gain observable en qualité de classement, en explicabilité ou en contrôle métier.

L'évaluation a été conduite en trois niveaux. Le premier niveau porte sur le modèle d'embeddings, c'est-à-dire la capacité du SentenceTransformer fine-tuné à retrouver les paires offre-profil pertinentes dans un corpus de test. Le deuxième niveau porte sur l'ablation du moteur de recommandation : une configuration *pgvector seul* est comparée à une configuration *pgvector + graphe*. Le troisième niveau porte sur les limites opérationnelles observées, notamment la stabilité de la connexion Neo4j et la portée réelle des métriques utilisées.

Les résultats présentés dans ce chapitre proviennent de deux artefacts exécutables : le benchmark d'embeddings sauvegardé dans `outputs/evaluation/embedding_benchmark.json` et le notebook d'ablation `notebooks/10_Evaluation_pgvector_vs_graph.ipynb`. Les sorties de ce notebook sont exportées dans `outputs/evaluation/pgvector_vs_graph` et les figures dans `rapport/figures/generated/evaluation`. Cette traçabilité est importante : les nombres présentés ici ne sont pas des valeurs théoriques, mais des sorties produites par le pipeline local.

4.2 Données et protocole expérimental

Le système dispose de 7 861 offres normalisées et de 1 105 profils candidats. Le benchmark d'embeddings utilise le jeu de test issu de la construction des paires contrastives offre-profil, soit 473 requêtes de test. Chaque requête correspond à une description structurée d'offre ou de profil, et le corpus contient les textes enrichis utilisés pour l'apprentissage contrastif. Les métriques calculées sont celles de la recherche d'information : Precision@K, Recall@K, NDCG@K et MRR@K.

Pour l'ablation *pgvector* contre *pgvector + graphe*, le protocole est différent. Un échantillon de 60 candidats est tiré aléatoirement avec la graine 42. Pour chaque candidat, pgvector retourne un pool initial de 30 offres candidates. Deux classements sont alors comparés :

- **pgvector seul** : l'ordre des offres est conservé tel qu'il est retourné par la recherche sémantique et lexicale dans PostgreSQL/pgvector ;
- **pgvector + graphe** : le même pool initial est enrichi par Neo4j, puis rerangé par le score hybride combinant similarité sémantique, compatibilité de compétences, compatibilité de niveau NCF et alignement sectoriel.

Cette comparaison est volontairement réalisée sans génération LLM. Le modèle génératif OpenRouter intervient dans le chatbot pour formuler la réponse finale, mais il n'est pas mobilisé dans l'ablation. Cette séparation évite de confondre deux phénomènes : la qualité du classement des offres et la qualité rédactionnelle de la réponse.

La pertinence utilisée pour calculer les métriques d'ablation est un proxy construit à partir des méta-données normalisées : chevauchement lexical entre le profil et l'offre, proximité sectorielle et compatibilité de niveau NCF. Cette approximation est acceptable pour comparer deux variantes internes sur le même échantillon, mais elle ne remplace pas une annotation humaine. Les résultats doivent donc être interprétés comme une mesure comparative de l'effet du graphe, non comme une mesure définitive de satisfaction utilisateur.

4.3 Évaluation du modèle d'embeddings

Le premier résultat concerne l'intérêt du fine-tuning du SentenceTransformer. Le tableau 4.1 présente les performances du modèle fine-tuné par rapport à plusieurs modèles généralistes ou multilingues. Le modèle spécialisé domine nettement les alternatives sur les métriques de retrieval.

TABLE 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L'écart est substantiel : le modèle spécialisé retrouve beaucoup plus souvent le document pertinent dans les dix premiers résultats. Le NDCG@10 passe également de 0.1980 à 0.6336, ce qui indique non seulement une meilleure récupération, mais aussi un meilleur ordre des résultats pertinents. La latence moyenne du modèle spécialisé reste contenue à 32.77 ms par phrase, ce qui est compatible avec une indexation ou une recherche interactive locale.

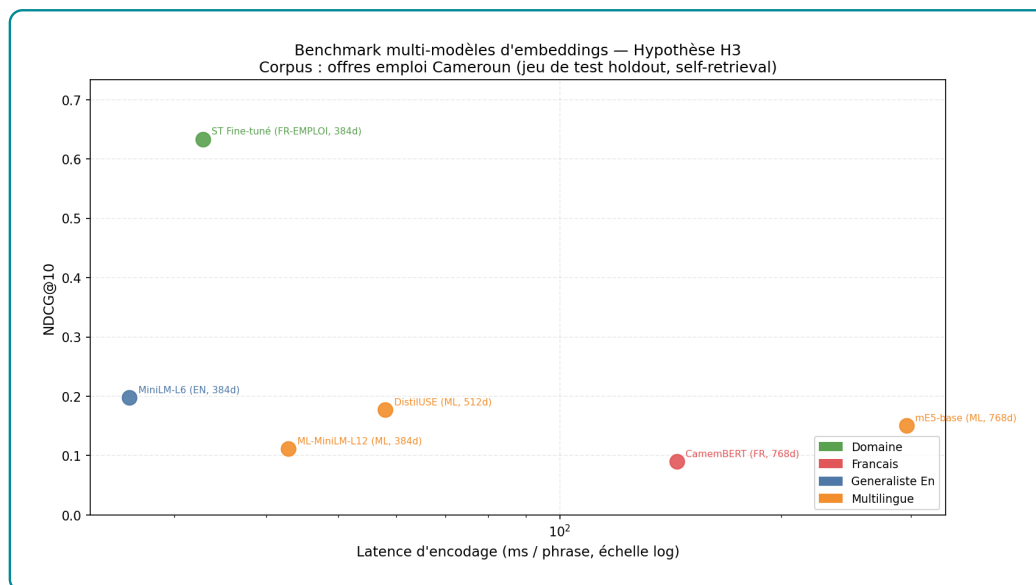


FIGURE 4.1 – Comparaison des modèles d'embeddings sur le jeu de test

Ce résultat valide le choix d'un encodeur adapté au domaine. Il montre aussi pourquoi une base vectorielle généraliste ne suffit pas : le vocabulaire emploi-compétences contient des formulations longues, des intitulés hybrides, des niveaux de formation et des compétences implicites. Le fine-tuning contrastif rapproche ces formulations dans l'espace vectoriel et améliore directement le retrieval.

4.4 Ablation pgvector seul contre pgvector + graphe

Le coeur de l'évaluation concerne l'apport de Neo4j. Le notebook d'ablation a demandé l'évaluation de 60 candidats. Sur cet échantillon, 55 requêtes ont été évaluées avec succès ; 5 requêtes ont échoué en raison d'une erreur interne Neo4j de type `CursorException` sur certaines relations. Cette information doit être conservée dans l'analyse : elle montre que la qualité d'un système GraphRAG ne dépend pas seulement du raisonnement algorithmique, mais aussi de l'intégrité physique et logique de la base graphe.

TABLE 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.2509	0.4599	0.3889	0.5602	0.9273
pgvector + graphe	0.3091	0.5557	0.4681	0.4647	0.7818
Différence	+0.0582	+0.0958	+0.0792	-0.0955	-0.1455

Les résultats sont instructifs parce qu'ils ne sont pas uniformément favorables au graphe. L'ajout de Neo4j améliore Precision@5, Recall@10 et NDCG@10. Autrement dit, lorsque l'on considère les cinq ou dix premiers résultats, l'enrichissement graphe permet de remonter davantage d'offres jugées pertinentes par le proxy métier. Le Recall@10 progresse de 0.4599 à 0.5557, soit un gain de 0.0958. Le NDCG@10 augmente également de 0.3889 à 0.4681, ce qui indique une meilleure organisation globale du top 10.

En revanche, le MRR@10 diminue de 0.5602 à 0.4647 et la Precision@1 passe de 0.4182 à 0.3636. Cela signifie que le graphe ne place pas toujours la première offre pertinente aussi haut que le classement

vectorel initial. Ce résultat est important : le graphe améliore la couverture et la cohérence globale du classement, mais il peut dégrader le tout premier rang lorsque ses signaux structurels sont imparfaits, incomplets ou trop fortement pondérés.

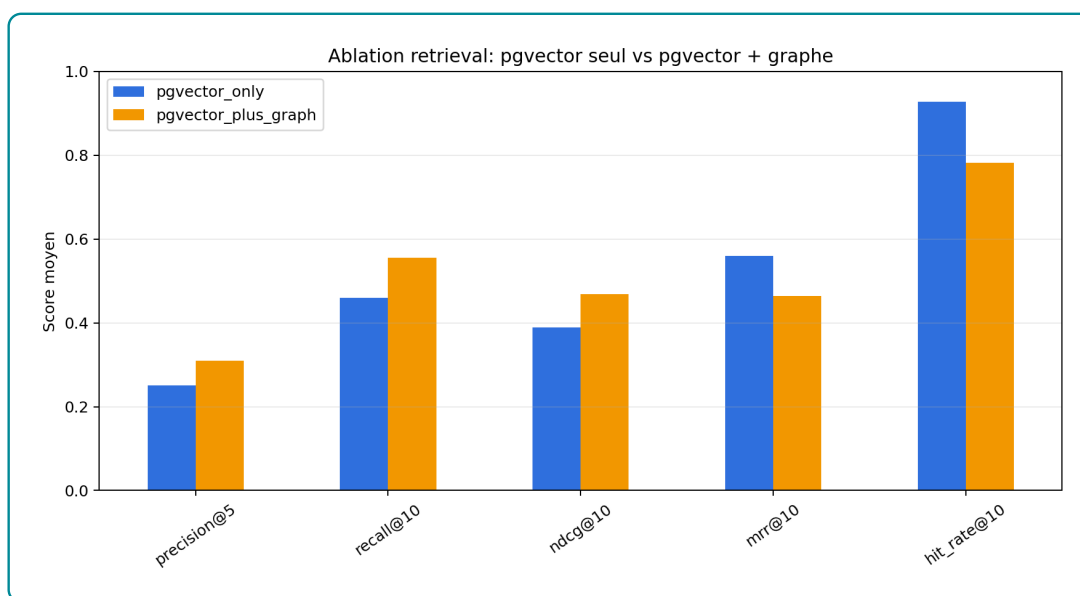


FIGURE 4.2 – Effet de l'enrichissement graphe sur les métriques de retrieval

La figure 4.2 confirme cette lecture. La configuration *pgvector + graphe* est supérieure sur les métriques orientées couverture, mais inférieure sur les métriques sensibles au premier rang. Dans un système de recommandation destiné au recrutement, cette distinction est opérationnellement importante. Si l'interface n'affiche qu'une seule offre, *pgvector* seul peut être plus robuste. Si l'interface affiche une liste courte accompagnée d'explications, le graphe devient plus utile car il enrichit le top 5 ou le top 10 avec des critères métier explicables.

4.5 Analyse du reranking par le graphe

Le reranking graphe déplace les offres dans le classement initial. Une offre qui était par exemple au rang 8 dans *pgvector* peut être remontée au rang 3 si elle présente une meilleure compatibilité NCF, un meilleur alignement sectoriel ou un meilleur score de compétences. Inversement, une offre très proche sémantiquement peut être rétrogradée si le niveau requis ou les signaux métier sont moins compatibles.

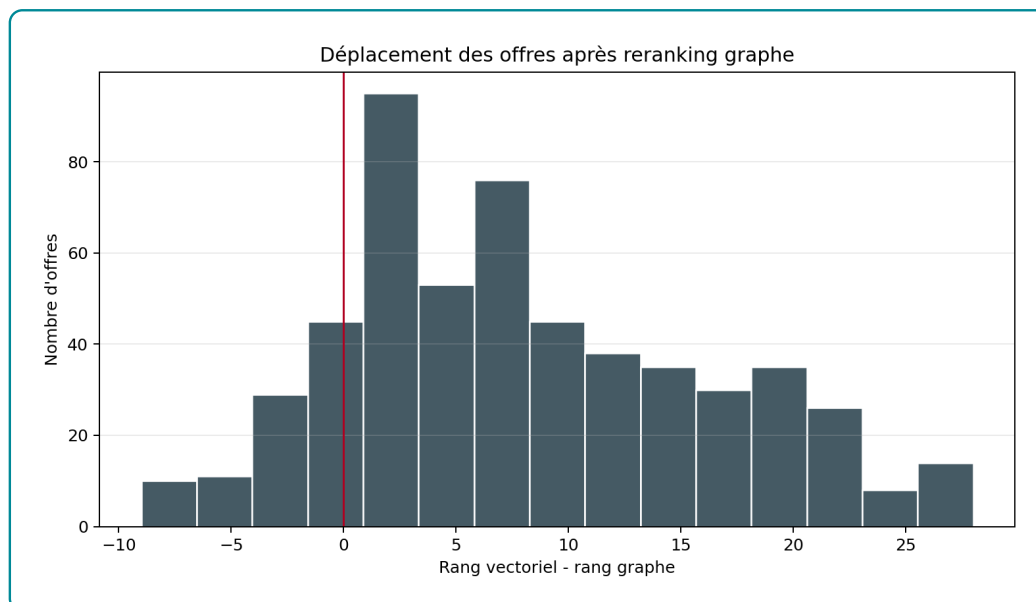


FIGURE 4.3 – Distribution des déplacements de rang après reranking par le graphe

Cette redistribution a deux effets. Le premier est positif : le système ne se limite plus à une proximité textuelle. Il peut corriger des cas où une offre ressemble lexicalement au profil mais ne correspond pas au niveau de formation ou au secteur recherché. Le second est plus problématique : si les relations du graphe sont incomplètes, le score structurel peut pénaliser de bonnes offres ou favoriser des offres seulement parce que leurs métadonnées sont mieux renseignées. Le graphe ajoute donc de l'information, mais il ajoute aussi une source d'erreur potentielle.

Ce résultat appelle une conclusion nuancée. Le graphe n'est pas une garantie automatique d'amélioration. Il améliore le système si les relations sont correctes, si les poids du score hybride sont bien calibrés et si l'objectif est de produire une liste explicable plutôt qu'un unique meilleur résultat. Dans le système actuel, l'apport est visible sur Recall@10 et NDCG@10, mais il reste insuffisamment stabilisé sur le rang 1.

4.6 Performance opérationnelle et stabilité

L'évaluation d'ablation a pris 38.7 secondes pour le cœur retrieval-reranking, hors coût complet de démarrage du notebook et de chargement du modèle. Le notebook exécuté par `nbconvert` a produit les résultats en environ 85 secondes en incluant le chargement du modèle SentenceTransformer, les connexions aux bases et l'export des figures. Cette durée reste acceptable pour une évaluation hors ligne, mais elle ne représente pas encore une latence utilisateur finale, car le chatbot complet ajoute la génération LLM via OpenRouter.

Le point critique est la stabilité Neo4j. Cinq candidats sur soixante ont échoué à cause d'une erreur interne `Neo.DatabaseError.Statement.ExecutionFailed` liée au page cache et à des relations corrompues. Le système a donc évalué 55 requêtes valides. Cette fragilité doit être traitée avant toute démonstration en production : il faut reconstruire ou vérifier la base Neo4j, contrôler les imports, et ajouter des tests d'intégrité sur les relations candidat-offre-compétence.

4.7 Interprétation par rapport aux hypothèses

Les résultats soutiennent partiellement l’hypothèse selon laquelle l’intégration d’un graphe améliore la recommandation. L’hypothèse est confirmée pour les métriques de couverture du top 10 : Recall@10 et NDCG@10 progressent lorsque Neo4j est utilisé pour enrichir et reranger les offres. En revanche, elle n’est pas confirmée pour le premier rang : Precision@1, MRR@10 et HitRate@10 diminuent. Le graphe apporte donc un gain structurel, mais ce gain n’est pas encore parfaitement aligné avec l’optimisation du meilleur résultat immédiat.

Sur le plan méthodologique, cette conclusion est plus utile qu’une affirmation générale. Elle montre où le graphe apporte de la valeur : explicabilité, couverture, prise en compte des niveaux et secteurs. Elle montre aussi où le système doit être amélioré : calibration des poids, complétude des relations, robustesse de Neo4j, et validation par annotation humaine.

4.8 Limites de l’évaluation

La principale limite est l’absence d’un jeu d’annotations humaines indiquant quelles offres sont réellement pertinentes pour chaque candidat. Le proxy utilisé combine les métadonnées disponibles, mais il ne capture pas toutes les dimensions du recrutement : motivation, expérience qualitative, disponibilité, préférences salariales, soft skills ou contraintes géographiques fines. Les métriques doivent donc être lues comme des indicateurs comparatifs internes.

La deuxième limite concerne le graphe lui-même. Les erreurs Neo4j observées indiquent que l’instance locale doit être auditée. Une base graphe partiellement corrompue ou incomplète peut biaiser l’évaluation du système hybride. Avant une évaluation finale à plus grande échelle, il faudra reconstruire la base, vérifier les nombres de nœuds et de relations, puis contrôler des chemins représentatifs : candidat–compétence, offre–compétence, offre–niveau, candidat–formation et métier–compétence.

La troisième limite concerne la génération LLM. Ce chapitre a volontairement isolé le retrieval et le reranking. La fidélité des réponses générées, la qualité des citations, la robustesse du critic et l’expérience utilisateur nécessitent une évaluation complémentaire, idéalement avec un protocole d’annotation humaine ou un juge LLM contrôlé par des critères explicites.

4.9 Synthèse du chapitre

L’évaluation confirme d’abord l’intérêt du SentenceTransformer fine-tuné : il dépasse nettement les modèles généralistes sur le jeu de test emploi-compétences. Elle montre ensuite que Neo4j apporte un gain réel sur les métriques de couverture et d’ordre global du top 10, mais qu’il dégrade le tout premier rang dans l’état actuel du système. La conclusion n’est donc pas que le graphe est automatiquement supérieur à pgvector. La conclusion correcte est que le graphe enrichit le classement lorsqu’on cherche une liste explicable et structurée, mais qu’il exige une meilleure qualité de relations et une calibration plus fine pour optimiser le top 1.

Ces résultats orientent directement les recommandations du chapitre suivant : renforcer l’intégrité Neo4j, construire un jeu d’évaluation annoté, ajuster les poids du score hybride, et distinguer dans l’interface les cas où le système privilégie la proximité sémantique immédiate et ceux où il privilégie la compatibilité métier structurée.

DISCUSSION ET RECOMMANDATIONS

Conclusion générale

Bibliographie

- [1] Dale T. MORTENSEN et Christopher A. PISSARIDES. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](https://doi.org/10.2307/2297896).
- [2] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [3] George A. AKERLOF. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](https://doi.org/10.2307/1879431).
- [4] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](https://doi.org/10.2307/1882010).
- [5] Gary S. BECKER. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [6] David H. AUTOR. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](https://doi.org/10.1007/s12651-013-0128-z).
- [7] EUROPEAN COMMISSION. *ESCO : European Skills, Competences, Qualifications and Occupations*. 2024. URL : <https://esco.ec.europa.eu/> (visit  le 04/05/2026).
- [8] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [9] Jacob DEVLIN et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [10] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [11] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [12] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

- [13] Nils REIMERS et Iryna GUREVYCH. “Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks”. In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.
- [14] Edward J. HU et al. “LoRA : Low-Rank Adaptation of Large Language Models”. In : *arXiv preprint arXiv :2106.09685* (2021). URL : <https://arxiv.org/abs/2106.09685>.
- [15] Tim DETTMERS et al. “QLoRA : Efficient Finetuning of Quantized LLMs”. In : *Advances in Neural Information Processing Systems* 36 (2023). URL : https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- [16] Yu A. MALKOV et D. A. YASHUNIN. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [17] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. “Billion-scale Similarity Search with GPUs”. In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [18] PGVECTOR CONTRIBUTORS. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [19] Aidan HOGAN et al. “Knowledge Graphs”. In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](https://doi.org/10.1145/3447772).
- [20] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : [10.2200/S01045ED1V01Y202009AIM046](https://doi.org/10.2200/S01045ED1V01Y202009AIM046).
- [21] Patrick LEWIS et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [22] Michael J. PAZZANI et Daniel BILLSUS. “Content-Based Recommendation Systems”. In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [23] Anis REDJDAL. “Modèles de langage au service de la recommandation basée sur des sessions”. Mémoire de maîtrise, PolyPublie. Mém. de mast. Polytechnique Montréal, 2024. URL : <https://publications.polymtl.ca/59177/>.
- [24] Yehuda KOREN, Robert BELL et Chris VOLINSKY. “Matrix Factorization Techniques for Recommender Systems”. In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [25] Balázs HIDASI et al. “Session-based Recommendations with Recurrent Neural Networks”. In : *arXiv preprint arXiv :1511.06939* (2015). URL : <https://arxiv.org/abs/1511.06939>.
- [26] Jing LI et al. “Neural Attentive Session-based Recommendation”. In : *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 2017, p. 1419-1428. DOI : [10.1145/3132847.3132926](https://doi.org/10.1145/3132847.3132926).
- [27] Petar VELIČKOVIĆ et al. “Graph Attention Networks”. In : *International Conference on Learning Representations*. 2018.

- [28] Xiangnan HE et al. “LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation”. In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [29] Wang-Cheng KANG et Julian MCAULEY. “Self-Attentive Sequential Recommendation”. In : *2018 IEEE International Conference on Data Mining*. 2018, p. 197-206. DOI : [10.1109/ICDM.2018.00035](https://doi.org/10.1109/ICDM.2018.00035).
- [30] Gabriel de Souza Pereira MOREIRA et al. “Transformers4Rec : Bridging the Gap between NLP and Sequential/Session-Based Recommendation”. In : *Proceedings of the 15th ACM Conference on Recommender Systems*. 2021, p. 143-153. DOI : [10.1145/3460231.3474255](https://doi.org/10.1145/3460231.3474255).
- [31] Robin BURKE. “Hybrid Recommender Systems : Survey and Experiments”. In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [32] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](https://doi.org/10.1007/978-0-387-85820-3).
- [33] Nimeesha V. *From Stale Searches to Smart Agents : The Rise of Agentic RAG*. Article Medium consulte pour l’intuition generale sur l’Agentic RAG. 2024. URL : <https://medium.com/@nimeeshav02/from-stale-searches-to-smart-agents-the-rise-of-agentic-rag-9e9fe950bbfd>.
- [34] Shunyu YAO et al. “ReAct : Synergizing Reasoning and Acting in Language Models”. In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.
- [35] Akari ASAI et al. “Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection”. In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [36] Shahul ES et al. “RAGAS : Automated Evaluation of Retrieval Augmented Generation”. In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [37] Nandan THAKUR et al. “BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models”. In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [38] Niklas MUENNIGHOFF et al. “MTEB : Massive Text Embedding Benchmark”. In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.

ANNEXES

TABLE A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	<code>OffreEmploi</code> , <code>Employeur</code> , <code>Localisation</code>	<code>Candidat</code> , <code>Secteur</code> , <code>PUBLIEE_PAR</code> relie une offre à son employeur ; <code>DANS_SECTEUR</code> la rattache à un secteur ; <code>LOCALISEE_A</code> indique la ville ou zone d'exercice ; <code>A_NIVEAU</code> relie le candidat à son niveau de formation.
Compétences et métiers	<code>Compétence</code> , <code>GroupeCompétences</code> , <code>GroupeISCO</code>	<code>Métier</code> , <code>NECESSITE</code> relie un métier aux compétences attendues ; <code>PARTIE_DE</code> rattache une compétence à son groupe ; <code>CLASSIFIE_DANS</code> relie un métier à un groupe ISCO ; <code>PLUS_LARGE_QUE</code> représente les relations hiérarchiques entre compétences.
Référentiel MEPC	<code>GrandGroupeMEPC</code> , <code>SousGroupeMEPC</code> , <code>GroupeBaseMEPC</code>	<code>CONTIENT</code> représente la hiérarchie interne de la nomenclature ; <code>ALIGNE_AVEC</code> rapproche les groupes MEPC des groupes ISCO ; <code>CORRESPOND_MEPC</code> rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	<code>NiveauFormationNCF</code> , <code>GrandDomaineNCF</code> , <code>DomaineSpécialiséNCF</code> , <code>DomaineDétailléNCF</code>	<code>CONTIENT</code> représente la hiérarchie formation ; <code>REQUIERT_NIVEAU_NCF</code> relie une offre au niveau demandé ; <code>A_FORMATION</code> rattache un candidat à son domaine de formation ; <code>PREPARE_POUR</code> relie un domaine de formation aux métiers visés.
Recommandation	<code>OffreEmploi</code> , <code>Compétence</code> , <code>DomaineDétailléNCF</code>	<code>Candidat</code> , <code>Métier</code> , Les chemins candidat-compétence-offre permettent d'identifier les compétences acquises et manquantes ; les chemins offre-niveau-formation servent à contrôler la compatibilité de qualification ; les chemins formation-métier soutiennent la proposition de montée en compétences.

Source : Auteur

TABLE A.2 – Volume d'entités indexées dans la base vectorielle pgvector

Type d'entité	Nombre	Source principale
OFFRE_EMPLOI	15 722	Scraping KmerAI
COMPETENCE	13 939	Référentiel ESCO v1.1
METIER	3 039	Référentiel ESCO v1.1
CANDIDAT	1 105	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total	34 215	

Source : Auteur

Table des matières

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	vi
Liste des tableaux	vii
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
 I Cadre théorique et conceptuel	 5
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	6
1.1 Marché de l'emploi, information imparfaite et appariement des compétences	6
1.1.1 Le marché du travail comme marché d'appariement	6
1.1.2 Asymétrie d'information et signalement des compétences	7
1.1.3 Capital humain, compétences et inadéquation	7
1.1.4 Transformation numérique et demande de compétences	8
1.1.5 Compétences, formation et besoin de standardisation	8
1.2 Les fondements des LLMs	9
1.2.1 Définition et évolution historique des LLMs en NLP	10
1.2.1.1 Les approches statistiques et le sac-de-mots (Bag-of-Words)	10
1.2.1.2 Les représentations vectorielles denses Word2Vec (2013) :	11
1.2.1.3 Les réseaux récurrents et l'introduction de l'attention (2014-2016)	11
1.2.1.4 Le Transformer et l'ère des LLMs (2017-aujourd'hui)	11
1.2.2 Le concept d'embeddings et représentations vectorielles	11
1.2.2.1 Le rôle des embeddings dans l'architecture Transformer	12

1.2.3	L'architecture Transformer et le mécanisme d'attention	12
1.2.3.1	Le mécanisme d'auto-attention	12
1.2.3.2	L'attention multi-têtes	13
1.2.3.3	Composants internes d'un bloc Transformer	13
1.3	Généralité sur le fine-tuning des modèles de LLMs	14
1.3.1	Pré-entraînement, adaptation et spécialisation	15
1.3.2	Les principales formes de fine-tuning	15
1.3.2.1	Fine-tuning supervisé	15
1.3.2.2	Instruction tuning	15
1.3.2.3	Fine-tuning contrastif pour les embeddings	15
1.3.2.4	Adaptation légère des paramètres	16
1.3.3	Application au matching emploi-compétences	16
1.3.4	Risques et précautions méthodologiques	16
1.4	Généralité sur des BD vectorielles et orientées graphes	16
1.4.1	Structure générale d'une base de données vectorielle	17
1.4.2	Principe de la recherche sémantique dans une BD vectorielle	17
1.4.3	Structure générale d'une base de données orientée graph	18
1.4.4	Techniques de description et d'analyse des graphes de connaissances	18
1.4.5	Synergie LLM + Graphes de Connaissances	19
1.5	Généralité sur les systèmes de recommandation	20
1.5.1	Filtrage basé sur le contenu (<i>Content-Based Filtering</i>)	20
1.5.2	Des méthodes traditionnelles aux modèles séquentiels et neuronaux	21
1.5.3	Apports des graphes et des GNN dans la recommandation	21
1.5.4	Transformers et modèles de langage pour la recommandation	22
1.5.5	Vers une recommandation hybride orientée recrutement et montée en compétences	22
1.6	Génération Augmentée par Récupération (RAG) et GraphRAG	23
1.7	Agentic RAG	25
2	Approche méthodologique et données	28
2.1	Positionnement méthodologique	28
2.2	Sources de données et rôle dans le système	29
2.3	Préparation et normalisation des données	30
2.4	Alignement des référentiels métiers et formations	31
2.5	Choix du modèle d'embeddings et fine-tuning	31
2.6	Indexation vectorielle dans PostgreSQL et pgvector	32
2.7	Construction du graphe de connaissances Neo4j	32
2.8	Score hybride et logique de skill gap	32
2.9	Architecture Agentic RAG et orchestration LangGraph	33
2.10	Text2Cypher et rôle du LLM	33
2.11	Interfaces et traçabilité	34
2.12	Protocole d'évaluation prévu	34

2.13	Limites méthodologiques et précautions d'interprétation	34
2.14	Synthèse de l'approche méthodologique	35
II	Présentation des résultats	36
3	Implémentation du système de recommandation	37
3.1	Vue d'ensemble de l'implémentation	37
3.2	Implémentation de la chaîne ETL	38
3.3	Implémentation du fine-tuning SentenceTransformer	39
3.4	Implémentation de la base vectorielle pgvector	39
3.5	Implémentation du graphe de connaissances Neo4j	40
3.6	Implémentation du moteur de recommandation hybride	41
3.7	Implémentation du pipeline Text2Cypher	41
3.8	Implémentation du workflow agentique LangGraph	42
3.9	Implémentation du LLM final et du critic	43
3.10	API, CLI et interface Streamlit	43
3.11	Diagnostic d'intégration	44
3.12	Synthèse du chapitre	45
4	Évaluation de la performance du système	46
4.1	Objectif et logique générale de l'évaluation	46
4.2	Données et protocole expérimental	46
4.3	Évaluation du modèle d'embeddings	47
4.4	Ablation pgvector seul contre pgvector + graphe	48
4.5	Analyse du reranking par le graphe	49
4.6	Performance opérationnelle et stabilité	50
4.7	Interprétation par rapport aux hypothèses	51
4.8	Limites de l'évaluation	51
4.9	Synthèse du chapitre	51
5	Discussion et recommandations	52
	Conclusion générale	I
	Bibliographie	I
	Bibliographie	II
A	Annexes	V